JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 04 OF 12 - STUDY STEP FW-04

Spring Boot for Java Backend Engineers

From First Application to Auto-Configuration, Production Operations, and SDE-2 Incidents

BY

Vinay Reddy Kalluri

A mechanics-first, executable guide to Spring Boot application assembly, typed configuration, HTTP boundaries, operations, testing, delivery, upgrades, and production diagnosis.

SPRING BOOT | AUTO-CONFIGURATION | HTTP | ACTUATOR | OPERATIONS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `FW 03 - Spring Framework`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Build on Spring Framework mechanics to teach how Boot assembles, configures, tests, packages, observes, secures, and operates production Java services from beginner foundations through SDE-2 incident reasoning.

Readiness map

Decision	Evidence
START HERE	A starter or classpath change activates or removes application infrastructure; A condition, user bean, property source, or application type changes auto-configuration; A service needs stable HTTP, data, observability, availability, testing, packaging, or deployment boundaries; An interview requires failure, retry, shutdown, resource, and incident behavior rather than annotation recall.
PREPARE FIRST	FW-03 Spring Framework; JAVA-03 Maven and Gradle; JAVA-01 Java Foundations; Java 21 and a terminal for executable labs.
MOVE ON WHEN	Build, package, and trace a Spring Boot application from main method through readiness; Predict starter, BOM, plugin, condition, back-off, property precedence, binding, and profile behavior; Design stable REST, error, pagination, idempotency, outbound-client, data, and migration boundaries; Operate Actuator, metrics, traces, probes, graceful shutdown, containers, resources, AOT, and upgrades safely; Answer realistic SDE-2 Spring Boot interview and production-incident scenarios from evidence.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 03 - Spring Framework	YOU ARE HERE FW 04 - Spring Boot	FW 05 - Spring Boot and REST

A note from Vinay

Spring Boot removes setup, not responsibility. When behavior surprises you, walk backward from the condition report and effective configuration to the auto-configuration decision.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Learning Path and Spring Boot First Principles

Spring Boot turns a Spring application into a runnable, configurable, observable service with fewer explicit setup decisions. It does this by contributing configuration based on the classpath, existing beans, properties, and application type. It does not replace Spring Framework, remove the need for architecture, or make production behavior automatic.

This publication targets **Java 21 and Spring Boot 4.1.0**. Spring Boot 4.1 requires at least Java 17 and Spring Framework 7.0.8. Version-sensitive features are labeled, while the core mental models also apply to maintained Spring Boot 3.5 and 4.0 services.

The one model to keep in your head

TEXT

```
build dependencies + application code + external configuration
                          |
                          v
                  SpringApplication
           prepares Environment and context
                          |
                          v
              user configuration first
                          |
                          v
       conditional auto-configuration fills gaps
                          |
                          v
        runnable application + operational signals
```

Boot is an **application assembly layer**. Spring Framework still owns the container, dependency injection, proxying, transactions, MVC, validation, and events. Boot supplies conventions, dependency management, auto-configurations, executable packaging, configuration loading, Actuator integration, and test support.

Start from plain Spring

Without Boot, a servlet application may explicitly register MVC infrastructure, configure a server, create a JSON mapper, establish property sources, and package for deployment. Boot observes that the application is servlet-based and that relevant libraries are present, then registers sensible defaults if the application has not supplied replacements.

That sentence contains four interview-critical ideas:

1. **Classpath:** dependencies are inputs to behavior.
2. **Conditions:** auto-configuration is selected, not unconditional.
3. **Back-off:** user beans can replace specific defaults.
4. **Evidence:** the condition evaluation report explains the decision.

Boot versus Framework versus platform

Boundary	Responsibility
Java	objects, threads, exceptions, memory, language rules
Spring Framework	container, AOP, transactions, MVC, validation
Spring Boot	application startup, conditional assembly, configuration conventions, packaging, operations
Deployment platform	processes, containers, networking, secrets, probes, scaling

If a Kubernetes pod restarts, Boot did not scale the service. If a method transaction rolls back, Boot did not invent transaction semantics. If an actuator endpoint is exposed publicly, the deployment and security configuration still own that risk.

The running service

Examples use an order API:

```text
HTTP client
    |
    v
OrderController -> OrderService -> OrderRepository -> database
                         |
                         +-> PaymentClient -> remote provider
                         +-> metrics and logs
```

The core invariant is: one idempotency key creates at most one order. The service must also start deterministically, reject invalid configuration, expose safe health signals, shut down without dropping in-flight requests, and produce evidence when it fails.

The learning sequence

```text
first application -> build and startup -> package structure
        |
        v
auto-configuration -> external configuration -> typed properties
        |
        v
REST boundaries -> outbound calls -> data and migrations
        |
        v
Actuator -> observability -> availability and shutdown
        |
        v
testing -> packaging -> performance/upgrades -> incidents/interviews
```

Do not memorize annotations before understanding what they import. Do not discuss native images before you can diagnose a failed context. Do not add every Actuator endpoint to HTTP exposure before deciding who may read it.

A seven-question Boot trace

For every feature, ask:

1. Which dependency put the feature on the classpath?
2. Which user configuration registered first?
3. Which auto-configuration condition matched?
4. Which condition caused another candidate to back off?
5. Which external value won, and from which property source?
6. Which runtime boundary handles the request, thread, and failure?
7. Which test, endpoint, log, metric, or trace proves the answer?

Common myths corrected

- Boot is not a code generator and does not replace Spring Framework.
- A starter is a dependency descriptor, not a runtime container.
- Auto-configuration is conditional and can be inspected.
- `application.yml` is not always the highest-precedence configuration source.
- A healthy process is not necessarily ready for traffic.
- `@SpringBootTest` is not the correct default for every test.
- An executable jar is not automatically a production-safe container image.

Quick check

1. What responsibilities does Boot add to Framework?
2. What four inputs determine common auto-configuration decisions?
3. Why does a user bean make an auto-configuration back off?
4. How are liveness and readiness different?
5. What evidence should replace guessing about startup?

Practice

- **Foundation:** Draw the four runtime boundaries for a small REST service.
- **Foundation:** Explain starter, auto-configuration, and embedded server in separate sentences.
- **Interview Core:** Trace how adding a JDBC driver can change startup behavior.
- **Interview Core:** List three reasons a property in `application.yml` may not win.
- **SDE-2 Follow-up:** Define the evidence needed to approve a new Boot starter in a production service.

TRY IT | Readiness checkpoint

Continue when you can describe Boot as conditional application assembly, not magic, and can separate Spring behavior from deployment-platform behavior.

SDE-2 CORE CHAPTER

Chapter 2 - First Application and Project Anatomy

A useful first application is small enough to explain completely. Start with one build file, one application class, one controller, one configuration file, and one test.

Minimal Maven project

XML

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
</parent>

<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Boot 4.1 prefers the focused `spring-boot-starter-webmvc`; the older `spring-boot-starter-web` is deprecated in its favor. A maintained Boot 3 application commonly still uses `spring-boot-starter-web`.

Application entry point

JAVA

```java
package com.example.orders;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OrderApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
```

`SpringApplication.run` returns a configured application context. It prepares the environment, determines the application type, loads initializers and listeners, creates the context, registers configuration, refreshes the context, starts the web server when applicable, runs startup callbacks, and publishes lifecycle events.

First endpoint

JAVA

```java
package com.example.orders.api;

import java.time.Instant;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
final class StatusController {
    @GetMapping("/api/status")
    StatusResponse status() {
        return new StatusResponse("ready", Instant.now());
    }
}

record StatusResponse(String status, Instant observedAt) { }
```

The return value becomes JSON because MVC infrastructure and a JSON message converter are available. The annotation alone does not serialize anything; registered MVC components inspect it.

Expected response shape:

JSON

```json
{
  "status": "ready",
  "observedAt": "2026-07-30T12:00:00Z"
}
```

Run and package

BASH

```bash
./mvnw spring-boot:run
./mvnw clean verify
java -jar target/order-service-1.0.0.jar
```

The Maven wrapper pins the build tool used by developers and CI. The Boot Maven plugin repackages the application with its dependencies and launcher metadata. `java -jar` starts the same application entry point without requiring a separately installed servlet container.

Project anatomy

TEXT

```
order-service/
  pom.xml
  mvnw
  .mvn/wrapper/
  src/main/java/com/example/orders/OrderApplication.java
  src/main/java/com/example/orders/api/StatusController.java
  src/main/resources/application.yml
  src/test/java/com/example/orders/OrderApplicationTests.java
```

Keep the application class in a root package above application components. Its package becomes an important default for component, entity, and repository discovery.

What a starter does not do

A starter contributes a curated dependency graph. It does not guarantee that an auto-configuration matches, that an external server is reachable, or that production settings are correct. Inspect the dependency tree:

BASH

```
./mvnw dependency:tree
./gradlew dependencies
```

Do not declare versions for libraries managed by Boot without a tested reason. Overriding one component can produce a combination Boot did not test.

WATCH OUT | Common mistakes

- Placing the application class in a leaf package, so sibling components are not scanned.
- Adding both servlet and reactive web stacks accidentally.
- Copying dependency versions from random examples.
- Running from the IDE but never verifying the packaged jar.
- Treating a successful HTTP response as proof that readiness and shutdown are correct.

Interview angle

Interviewer: What does `@SpringBootApplication` give you?

Strong answer: It combines a configuration class, component scanning, and enablement of Boot auto-configuration. The package location defines default discovery boundaries. `SpringApplication.run` prepares the environment and refreshes the appropriate context; dependencies and conditions decide which infrastructure is registered.

Quick check

1. Why use a Maven or Gradle wrapper?
2. What does the Boot plugin add to packaging?
3. Why does the package of the application class matter?
4. What turns a controller return value into JSON?
5. Why inspect the dependency tree?

Practice

- **Foundation:** Create `/api/status` and verify it from the packaged jar.
- **Foundation:** Move the controller outside the root package and explain the resulting 404.
- **Interview Core:** Explain every line of the application class.
- **Interview Core:** Compare a starter, an auto-configuration, and the embedded server.
- **SDE-2 Follow-up:** Write a CI gate that proves tests, packaging, and process startup.

SDE-2 CORE CHAPTER

Chapter 3 - SpringApplication Startup and Lifecycle

Production diagnosis starts with a timeline. A Boot process can fail before logging is configured, while the environment is prepared, during bean creation, while the web server binds, in a startup runner, or after readiness begins.

Simplified startup sequence

TEXT

```
main
  -> create SpringApplication
  -> discover listeners and initializers
  -> prepare Environment and bind spring.main.*
  -> print banner and create ApplicationContext
  -> load configuration sources
  -> refresh context
       -> register definitions
       -> apply auto-configuration
       -> create non-lazy singletons
       -> start embedded server
  -> publish started event
  -> run ApplicationRunner / CommandLineRunner
  -> publish ready event
```

The exact internal calls are more detailed, but this sequence is sufficient to classify failures and avoid treating every startup error as a bean error.

Application type

Boot infers whether the application is non-web, servlet, or reactive from the classpath. Mixed web stacks make this decision harder to reason about. When a service is intentionally non-web:

JAVA

```java
SpringApplication application = new SpringApplication(BatchApplication.class);
application.setWebApplicationType(WebApplicationType.NONE);
application.run(args);
```

Use an explicit type only when it communicates real intent; do not use it to hide accidental dependencies.

Arguments and runners

JAVA

```java
@Component
final class CacheWarmup implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments arguments) {
        boolean verifyOnly = arguments.containsOption("verify-only");
        // bounded startup work
    }
}
```

`ApplicationArguments` separates option and non-option arguments. `CommandLineRunner` receives raw strings. Multiple runners can implement `Ordered`, but complex ordering is a smell. A runner executes before the ready event; slow or unbounded work delays readiness and may exceed platform startup limits.

Failure phases

Symptom	Likely phase	First evidence
property binding error	environment/context preparation	origin and binding failure
`NoSuchBeanDefinitionException`	context refresh	dependency graph and condition report
port already in use	server startup	bind exception and configured port
process starts then exits	runner or non-web lifecycle	exit code and final exception
ready then fails traffic	runtime dependency or request path	health, metrics, traces, logs

Startup events are signals, not business messages

Boot publishes lifecycle events such as starting, environment prepared, context initialized, started, ready, and failed. Some occur before the application context exists, so ordinary bean listeners cannot receive all of them. Register early listeners on `SpringApplication` or through supported discovery when truly needed.

Do not perform critical durable work only because a ready event fired. An event is in-process; the process can terminate afterward.

Exit codes

Command-line applications can map failures to process exit codes with `ExitCodeGenerator` or exception mappings. Operations systems depend on non-zero failure codes.

JAVA

```java
int code = SpringApplication.exit(context, () -> 12);
System.exit(code);
```

Avoid `System.exit` inside ordinary services and tests. Let the application boundary own process termination.

Measuring startup

Use `ApplicationStartup` for structured startup steps:

JAVA

```java
SpringApplication application = new SpringApplication(OrderApplication.class);
application.setApplicationStartup(new BufferingApplicationStartup(2048));
application.run(args);
```

With Actuator, the `startup` endpoint can expose buffered steps. This is diagnostic data; secure its exposure and choose buffer size deliberately.

WATCH OUT | Common mistakes

- Blocking forever inside a runner.
- Starting unmanaged threads that prevent shutdown.
- Assuming ready means every remote dependency is permanently healthy.
- Logging secrets while diagnosing environment preparation.
- Retrying a deterministic configuration error.
- adding lazy initialization globally to hide a broken bean graph.

Interview angle

Interviewer: The pod is killed before becoming ready. What do you inspect?

Strong answer: I locate the last completed startup phase from events and logs, separate environment binding, context refresh, server binding, and runner work, inspect the condition report for missing infrastructure, compare startup duration with probe thresholds, and reproduce the packaged artifact with the same effective configuration. I do not increase the probe timeout before determining whether work is bounded.

Quick check

1. When do runners execute relative to readiness?
2. Why can an early lifecycle listener not be an ordinary bean?
3. What decides application type?
4. How should a CLI failure reach its supervisor?
5. What does `ApplicationStartup` provide?

Practice

- **Foundation:** Add a runner that prints parsed option names.
- **Interview Core:** Classify five startup stack traces by phase.
- **Interview Core:** Make a startup task bounded and observable.
- **SDE-2 Follow-up:** Design startup for a cache that improves latency but is not required for correctness.

SDE-2 CORE CHAPTER

Chapter 4 - Application Structure and Scan Boundaries

Package structure is executable configuration in a Boot application. It affects component scanning, default entity/repository discovery, test configuration search, and accidental coupling.

A feature-oriented structure

TEXT

```
com.example.orders
  OrderApplication.java
  order/
    api/
    application/
    domain/
    persistence/
  payment/
    application/
    client/
  shared/
    configuration/
```

Feature-oriented packages keep one workflow discoverable. Layers can still exist inside each feature. A global `controller/service/repository` split often scatters a change across the repository and encourages dependencies between unrelated features.

What @SpringBootApplication combines

Conceptually:

JAVA

```java
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public @interface SpringBootApplication { }
```

`@SpringBootConfiguration` identifies primary Boot configuration. `@EnableAutoConfiguration` imports candidates conditionally. `@ComponentScan` finds application components from the annotated class's package by default.

Keep the root narrow

JAVA

```java
package com.example.orders;

@SpringBootApplication
public class OrderApplication { }
```

Do not put the class in `com.example`; that can scan every company library beneath the package. Do not put it in `com.example.orders.bootstrap` unless component scanning is configured deliberately.

Use explicit imports at module boundaries:

JAVA

```java
@Configuration(proxyBeanMethods = false)
@Import({OrderModuleConfiguration.class, PaymentClientConfiguration.class})
class ApplicationModules { }
```

This makes ownership reviewable and reduces accidental registration.

Configuration classes should stay thin

Configuration describes assembly:

JAVA

```java
@Configuration(proxyBeanMethods = false)
class PaymentClientConfiguration {
    @Bean
    PaymentClient paymentClient(PaymentProperties properties,
                                RestClient.Builder builder) {
        return new PaymentClient(builder.baseUrl(properties.baseUrl()).build());
    }
}
```

Business decisions belong in application/domain classes. A large configuration class with branching business logic becomes difficult to test and reuse.

Default package and unnamed package

Never use Java's unnamed/default package. Boot cannot establish a safe scan boundary and may inspect every class from every jar. Explicit packages also make tests and modularization predictable.

Architecture checks

A mature service can test dependency direction with architecture tests or simple package rules:

TEXT

```
api -> application -> domain
               |
               v
       ports/interfaces
               ^
               |
     persistence/client adapters
```

Boot is used at the outer composition boundary. Domain objects should not require a running Boot application to enforce invariants.

WATCH OUT | Common mistakes

- Broad `scanBasePackages` strings that silently grow.
- A second `@SpringBootApplication` in test sources that changes discovery.
- Component-scanning third-party library packages.
- Placing entities or repositories outside expected roots without explicit configuration.
- Using package-private framework components across unrelated packages accidentally.

Interview angle

Interviewer: A component exists but is not injected. What do you verify?

Strong answer: I confirm it is registered, not merely annotated; check the primary configuration package and explicit scans/imports; verify profiles and conditions; look for duplicate test configuration; then inspect candidate type and qualifiers. I prefer correcting the module boundary over widening the scan to the company root.

Quick check

1. Which three concerns are combined by `@SpringBootApplication`?
2. Why is the application package a runtime boundary?
3. Why prefer feature-oriented packages?
4. When is explicit `@Import` clearer than scanning?
5. Why keep the domain independent of Boot?

Practice

- **Foundation:** Reorganize a controller/service/repository sample by feature.
- **Foundation:** Predict which packages a root application class scans.
- **Interview Core:** Diagnose a test that finds a different primary configuration.
- **SDE-2 Follow-up:** Define module dependency rules for orders, payments, and notifications.

SDE-2 CORE CHAPTER

Chapter 5 - Starters, Dependency Management, and Build Plugins

Boot build support solves three different problems: choosing capabilities, aligning versions, and packaging an application. Candidates often call all three a starter; that loses important failure modes.

Starters choose capabilities

XML

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

A starter is a curated dependency descriptor. It usually has little or no code. Its transitive libraries become classpath signals for auto-configuration.

The BOM aligns versions

The `spring-boot-dependencies` bill of materials manages compatible versions for Spring modules and many third-party libraries. The Maven parent imports it and adds plugin defaults. A project that cannot inherit the parent can import the BOM in `dependencyManagement`.

Gradle can use the Boot plugin with native platform support:

GROOVY

```groovy
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.0'
}

dependencies {
    implementation platform(org.springframework.boot.gradle.plugin.SpringBootPlugin.BOM_COORDINATES)
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

Do not also pin `spring-core`, Jackson, Tomcat, or logging versions casually. A local override can defeat a security fix or create binary incompatibility.

Plugins package and inspect

The Maven and Gradle Boot plugins can:

- create executable jars or wars;
- run the application during development;
- build OCI images through buildpacks;
- process AOT metadata;
- add build information for Actuator.

The plugin is not the dependency-management BOM, and the BOM does not produce an executable jar.

Read the graph before changing it

BASH

```
./mvnw dependency:tree -Dverbose
./mvnw help:effective-pom
./gradlew dependencies
./gradlew dependencyInsight --dependency jackson-databind
```

Ask who introduced a library, which version constraint won, and whether it is used at compile, runtime, or test time.

Exclusions and substitutions

To replace the default logging implementation, exclude deliberately and add one replacement. To switch the embedded server, use the focused runtime starter supported by the selected Boot line. Test startup, shutdown, access logs, TLS, compression, forwarded headers, and error behavior after the swap.

An exclusion is not a security policy. The excluded class may arrive through another path; verify the resolved graph and software bill of materials.

Version strategy

1. Pick a supported Boot line.
2. Let Boot manage the dependency set.
3. Record justified overrides with owner and removal condition.
4. Use dependency and vulnerability automation.
5. Upgrade maintenance releases continuously.
6. Treat major/minor upgrades as behavior changes with migration tests.

WATCH OUT | Common mistakes

- Adding every starter "just in case," increasing startup surface and CVEs.
- Depending on both blocking and reactive stacks accidentally.
- Confusing a managed version with a direct dependency.
- Using `-DskipTests` as a universal CI shortcut without understanding other build phases.
- Producing a thin jar locally but deploying an executable-jar command.
- Overriding Spring Framework independently of Boot.

Interview angle

Interviewer: A CVE scanner reports a transitive library. What do you do?

Strong answer: I identify the introducing path and resolved version, check whether the vulnerable code is reachable, prefer a supported Boot maintenance release that aligns the dependency set, and use a temporary override only with compatibility tests and a removal plan. I verify the packaged artifact and SBOM, not only the source build file.

Quick check

1. Starter versus BOM versus plugin?
2. What does an effective POM reveal?
3. Why can one version override be risky?
4. What should be tested after swapping the server?
5. Why inspect the packaged dependency graph?

Practice

- **Foundation:** Explain each transitive dependency in a minimal web service.
- **Interview Core:** Remove an unnecessary starter and compare the condition report.
- **Interview Core:** Diagnose a version conflict with `dependencyInsight`.
- **SDE-2 Follow-up:** Write an emergency-override policy for a critical transitive CVE.

SDE-2 CORE CHAPTER

Chapter 6 - Auto-Configuration, Conditions, and Back-Off

Auto-configuration is ordinary Spring configuration selected by evidence. Boot imports candidate classes, evaluates their conditions, and registers beans only when the application has not already provided the relevant capability.

A representative auto-configuration

JAVA

```java
@AutoConfiguration
@ConditionalOnClass(PaymentClient.class)
@EnableConfigurationProperties(PaymentProperties.class)
class PaymentClientAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    PaymentClient paymentClient(PaymentProperties properties,
                                RestClient.Builder builder) {
        return new PaymentClient(
                builder.baseUrl(properties.baseUrl().toString()).build());
    }
}
```

Read it as a decision:

TEXT

```
PaymentClient class present?
        |
        +-- no -> skip configuration
        |
        +-- yes -> PaymentClient bean already present?
                       |
                       +-- yes -> back off
                       +-- no  -> bind properties and create default
```

Common condition types

Condition	Question
`@ConditionalOnClass`	Is a library capability present?
`@ConditionalOnMissingClass`	Is a library absent?
`@ConditionalOnBean`	Did the application register a prerequisite?
`@ConditionalOnMissingBean`	Has the application already supplied a replacement?
`@ConditionalOnProperty`	Is a feature property present or equal to a value?
`@ConditionalOnWebApplication`	Is this servlet/reactive web context?
`@ConditionalOnResource`	Does a resource exist?

Conditions are evaluated during registration. Bean conditions see definitions processed so far, which is why auto-configuration is applied after user configuration. Do not make ordering accidental.

Back-off is local

Providing one `ObjectMapper`, `DataSource`, or `RestClient.Builder` may cause some defaults to back off while related auto-configurations remain. Replacing one bean does not disable an entire subsystem unless its conditions say so.

JAVA

```java
@Bean
PaymentClient paymentClient(RestClient.Builder builder) {
    return new PaymentClient(builder.baseUrl("https://sandbox.example").build());
}
```

Prefer a narrow replacement. Excluding a whole auto-configuration can remove supporting infrastructure you still need.

Inspect decisions

Run with the debug property or inspect the condition evaluation report:

BASH

```bash
java -jar app.jar --debug
```

The report groups positive matches, negative matches, unconditional classes, and exclusions. A negative match is not automatically an error; it often explains a capability not requested by the application.

Actuator's `conditions` endpoint can expose similar data at runtime when deliberately enabled and secured.

Excluding auto-configuration

JAVA

```java
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
class ToolApplication { }
```

Or:

PROPERTIES

```properties
spring.autoconfigure.exclude=\
org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
```

Use exclusion when the capability is genuinely not part of the application, not as the first fix for incomplete database configuration.

WATCH OUT | Common mistakes

- Assuming every classpath library creates beans.
- Declaring a replacement with the wrong type so back-off does not trigger.
- Depending directly on an auto-configuration class as public application API.
- Using `@ConditionalOnProperty` for a collection and expecting complex matching.
- Adding `@Order` to auto-configuration instead of supported before/after metadata.
- Reading only startup logs and ignoring the condition report.

Interview angle

Interviewer: A `DataSource` appears locally but not in production. How do you diagnose it?

Strong answer: I compare resolved dependencies, effective properties with origins, application type, user bean definitions, and the condition report. An embedded database may satisfy local classpath conditions while production expects an external URL. I make the dependency and configuration contract explicit and add a context test for both environments.

Quick check

1. When does auto-configuration apply relative to user configuration?
2. What does back-off mean?
3. Why can replacing one bean preserve the rest of a subsystem?
4. What does a negative match prove?
5. When is exclusion justified?

Predict and debug

Predict: A custom `PaymentClient` bean exists before the auto-configuration above. The missing-bean condition fails, so the default client is not registered.

Debug: Two clients exist because the custom bean returns a broader interface while the condition checks the implementation class. Align the public bean type/condition contract and test it with `ApplicationContextRunner`.

Practice

- **Foundation:** Explain a five-line condition report entry in plain language.
- **Interview Core:** Replace one auto-configured bean without excluding the subsystem.
- **Interview Core:** Test enabled, disabled, missing-class, and user-override cases.
- **SDE-2 Follow-up:** Diagnose a classpath-dependent local/production difference.

SDE-2 CORE CHAPTER

Chapter 7 - Custom Auto-Configuration and Starters

Application teams should use explicit configuration first. Custom auto-configuration is appropriate for a reusable library that must integrate with many Boot applications while allowing each application to replace defaults.

Separate library responsibilities

TEXT

```
acme-audit-core
  domain API and implementation

acme-audit-spring-boot
  configuration properties and auto-configuration

acme-audit-spring-boot-starter
  dependency descriptor for consumers
```

Small teams may combine the last two, but the conceptual separation remains: runtime implementation, conditional assembly, dependency convenience.

A safe auto-configuration

JAVA

```java
@AutoConfiguration
@ConditionalOnClass(AuditSink.class)
@ConditionalOnProperty(
        prefix = "acme.audit",
        name = "enabled",
        havingValue = "true",
        matchIfMissing = true)
@EnableConfigurationProperties(AuditProperties.class)
public class AuditAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    AuditSink auditSink(AuditProperties properties) {
        return new HttpAuditSink(properties.endpoint(), properties.timeout());
    }
}
```

Register it in:

TEXT

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

with the fully qualified class name. Do not rely on application component scanning to discover library auto-configuration.

Ordering without hidden coupling

Use `@AutoConfigureBefore`, `@AutoConfigureAfter`, or the corresponding names only when one auto-configuration consumes definitions from another. Ordering affects definition processing; it does not force bean creation order. Bean dependencies still determine creation.

Avoid depending on internal Boot auto-configuration implementation classes. Their public behavior and bean types are safer integration points.

Metadata and IDE support

The configuration processor produces metadata for typed properties. Document:

- property name and purpose;
- type and unit;
- default behavior;
- whether restart is required;
- security sensitivity;
- validation rules and deprecation path.

Do not use generated metadata as runtime validation. Binding and bean validation own correctness.

Test the decision matrix

JAVA

```java
private final ApplicationContextRunner runner = new ApplicationContextRunner()
        .withConfiguration(AutoConfigurations.of(AuditAutoConfiguration.class));

@Test
void backsOffForUserSink() {
    runner.withBean(AuditSink.class, InMemoryAuditSink::new)
          .run(context -> assertThat(context)
                  .hasSingleBean(AuditSink.class)
                  .getBean(AuditSink.class)
                  .isInstanceOf(InMemoryAuditSink.class));
}
```

Required cases include default match, property disabled, required class absent, required bean present/absent, user override, invalid properties, and relevant application types.

Compatibility contract

A starter becomes shared platform code. It needs:

- a supported Boot version range;
- dependency convergence tests;
- release notes and migration guidance;
- observability and failure semantics;
- no silent remote calls during startup unless explicitly contracted;
- a rollback plan if adoption breaks services.

WATCH OUT | Common mistakes

- Scanning library packages instead of using imports metadata.
- Making auto-configuration classes an application API.
- Creating a bean even when the user provided one.
- Fetching remote configuration in a bean constructor.
- Making a starter depend on unrelated databases, web stacks, or logging systems.
- Publishing without testing the packaged jar in a consuming application.

Interview angle

Interviewer: When would you build a company starter?

Strong answer: When multiple services need the same stable integration contract and defaults, not merely the same three annotations. I separate the core API from conditional Boot assembly, back off for user beans, expose validated typed configuration, test the condition matrix with `ApplicationContextRunner`, and version the starter like a platform product.

Quick check

1. Why not discover library configuration by component scan?
2. What belongs in a starter artifact?
3. What does auto-configuration ordering control?
4. Which cases belong in the condition test matrix?
5. Why is a company starter a long-term compatibility contract?

Practice

- **Foundation:** Write imports metadata for one auto-configuration.
- **Interview Core:** Add a user override and property-disable test.
- **Interview Core:** Design configuration metadata for timeout and endpoint values.
- **SDE-2 Follow-up:** Review a starter that performs a remote call during context refresh.

SDE-2 CORE CHAPTER

Chapter 8 - External Configuration, Precedence, Profiles, and Imports

External configuration lets one immutable artifact run in multiple environments. The difficult part is not YAML syntax; it is predicting which value wins, where it came from, and whether changing it is safe.

Values have origins and precedence

Boot creates an ordered environment. Later, higher-precedence sources override earlier ones. Important groups include:

TEXT

```
defaults and packaged config
        < profile-specific packaged config
        < external config files
        < profile-specific external config
        < environment variables / system properties
        < command-line arguments
        < test-specific overrides
```

This is a learning model, not a substitute for the complete official order. JSON application properties, servlet configuration, Devtools, and test annotations occupy defined positions too. When diagnosing, inspect the property source and origin instead of relying on a memorized partial list.

File locations

Boot loads `application.properties` or YAML from conventional classpath and external locations. External directories can override packaged defaults. Additional locations and names can be supplied, but changing location properties happens very early and can make startup depend on launch commands.

YAML

```yaml
server:
  port: 8080
payment:
  base-url: https://payments.example
  timeout: 2s
```

Environment variable names use relaxed binding conventions, for example `PAYMENT_BASEURL` or platform-specific mapped forms. Prefer testing the actual deployment mapping; shell and orchestrator behavior can differ.

Profiles select groups of configuration

YAML

```yaml
spring:
  profiles:
    group:
      production:
        - cloud
        - observability
```

A profile is a named configuration selection, not a security boundary. Avoid profiles such as `customer-a`, `customer-b`, and dozens of environment permutations. Use typed properties and deployment configuration for values; reserve profiles for coherent feature/configuration groups.

Profile activation cannot be defined inside a document that is itself conditional on that profile. Bootstrap decisions must be available before the document is selected.

Configuration imports

PROPERTIES

```properties
spring.config.import=optional:file:/etc/order-service/overrides.properties
```

Imports can bring additional configuration into the environment at a precise location. The `optional:` prefix controls whether absence is fatal. Use non-optional imports when missing configuration makes the service unsafe.

Config trees map files in a mounted directory to properties, which is useful for container secrets:

PROPERTIES

```properties
spring.config.import=configtree:/run/secrets/
```

Do not log the resulting secret values or expose them through Actuator.

Placeholders and random values

PROPERTIES

```properties
order.region=${ORDER_REGION:local}
order.instance-id=${HOSTNAME:unknown}
```

Defaults are useful only when they are safe. A default production database password or payment endpoint is not safe. `RandomValuePropertySource` is useful for tests and non-secret identifiers; it is not a secret-management system.

Diagnose one value

1. Write the canonical property name.
2. List all candidate sources.
3. Identify active profiles and imported documents.
4. Inspect the winning source and origin.
5. Confirm the bound target and conversion.
6. Remove accidental duplicates.

Actuator `env` and `configprops` endpoints sanitize values by default, but endpoint access still requires a security decision.

WATCH OUT | Common mistakes

- Assuming YAML wins over environment variables.
- Using profiles as tenant or feature-flag storage.
- Making a critical config import optional.
- Committing secrets to source control.
- Defining the same property in many places without ownership.
- Copying a property from a different Boot version.
- Assuming configuration reload occurs automatically.

Interview angle

Interviewer: Production ignores the value in `application.yml`. Why?

Strong answer: I inspect active profiles, external files and imports, environment/system/command-line values, relaxed name mapping, and the property origin. The packaged YAML is only one source. I also verify the value bound to the target type and whether the deployment injects an empty or malformed value.

Quick check

1. Why are value origin and precedence inseparable?
2. What is a profile appropriate for?
3. What does `optional:` change for an import?
4. Why is a random property not a secret?
5. Does Boot reload changed files automatically?

Practice

- **Foundation:** Predict winners for packaged, external, environment, and CLI values.
- **Foundation:** Convert three properties to environment-variable names.
- **Interview Core:** Design a fail-fast secret import.
- **Interview Core:** Replace twelve environment profiles with typed configuration.
- **SDE-2 Follow-up:** Produce a configuration provenance runbook that never prints secrets.

SDE-2 CORE CHAPTER

Chapter 9 - Configuration Properties, Binding, Validation, and Secrets

`@Value` is acceptable for one isolated value. A service capability with multiple related settings deserves a typed configuration object with validation, units, defaults, and ownership.

Immutable typed configuration

JAVA

```java
@ConfigurationProperties("payment")
@Validated
public record PaymentProperties(
        @NotNull URI baseUrl,
        @NotNull @DurationMin(millis = 50) Duration connectTimeout,
        @NotNull @DurationMin(millis = 100) Duration readTimeout,
        @Min(1) @Max(20) int maxConnections) {
}
```

Register it explicitly:

JAVA

```java
@Configuration(proxyBeanMethods = false)
@EnableConfigurationProperties(PaymentProperties.class)
class PaymentConfiguration { }
```

or scan a narrow application-owned package with `@ConfigurationPropertiesScan`.

Relaxed binding

The canonical property is kebab case:

PROPERTIES

```properties
payment.base-url=https://payments.example
payment.connect-timeout=250ms
payment.read-timeout=2s
payment.max-connections=8
```

Boot can bind common variants from environment sources. Do not depend on ambiguous acronyms or undocumented conversions. Keep names stable and generate metadata.

Conversion and units

Boot converts durations and data sizes:

JAVA

```java
@ConfigurationProperties("upload")
record UploadProperties(DataSize maxFileSize, Duration timeout) { }
```

Always include explicit units in configuration such as `10MB` and `750ms`. Bare numbers can depend on annotation/default-unit conventions and are easy to misread.

Fail fast on unsafe configuration

Validation should reject impossible state during startup:

- blank endpoint;
- negative timeout;
- zero pool size;
- unsupported region;
- retry count outside a tested bound.

Cross-field rules need a compact constructor or class-level validator:

JAVA

```java
public PaymentProperties {
    if (readTimeout.compareTo(connectTimeout) < 0) {
        throw new IllegalArgumentException(
                "payment.read-timeout must not be shorter than connect-timeout");
    }
}
```

Do not validate remote reachability in a constructor. That turns transient network failure into context creation failure and may leak credentials.

Secrets are references and values with stricter handling

Configuration binding does not make a value safe. Secrets require:

- delivery from an approved store or mounted secret;
- least-privilege access;
- rotation strategy;
- redaction from logs, heap dumps, endpoints, and error messages;
- no default fallback that silently uses an unsafe credential.

Prefer a credential-provider abstraction when values rotate during process lifetime. An immutable bound string cannot refresh itself.

@Value versus properties

`@Value`	`@ConfigurationProperties`
one local scalar	cohesive capability settings
string expression support	structured binding and metadata
easy to scatter	central ownership and validation
weaker refactoring	typed nested models

Test binding directly

JAVA

```java
new ApplicationContextRunner()
    .withUserConfiguration(PaymentConfiguration.class)
    .withPropertyValues(
        "payment.base-url=https://payments.example",
        "payment.connect-timeout=250ms",
        "payment.read-timeout=2s",
        "payment.max-connections=8")
    .run(context -> assertThat(context)
        .hasSingleBean(PaymentProperties.class));
```

Add invalid and missing-value cases. A happy-path context test alone does not prove the safety boundary.

WATCH OUT | Common mistakes

- Scattering related values across fields with `@Value`.
- Using strings for durations and parsing them repeatedly.
- Providing unsafe defaults for credentials or production endpoints.
- Expecting bean validation without the validation dependency/infrastructure.
- Logging an entire properties object containing secrets.
- Assuming a mounted secret rotates inside an already bound object.

Interview angle

Interviewer: Why did a typo in a property start the service with a default?

Strong answer: The property did not bind to the canonical field, and the target allowed a default. I add metadata, fail-fast validation for required values, a binding test using the deployment name, and an ownership rule that rejects unknown or obsolete properties where practical. I do not solve it by printing all configuration.

Quick check

1. When is `@Value` sufficient?
2. Why include units in configuration?
3. What belongs in startup validation?
4. Why should a constructor avoid remote reachability checks?
5. What changes when a secret rotates?

Practice

- **Foundation:** Convert five `@Value` fields into one record.
- **Interview Core:** Add valid, missing, and cross-field-invalid binding tests.
- **Interview Core:** Design redaction for a properties object.
- **SDE-2 Follow-up:** Support credential rotation without restarting every instance simultaneously.

SDE-2 CORE CHAPTER

Chapter 10 - REST API Boundaries, Validation, and Errors

A Boot REST API is still a Spring MVC application. Boot configures the dispatcher, server, message converters, validation integration, and error infrastructure; the application must define a stable HTTP contract.

Request flow

TEXT

```
socket -> embedded server -> filter chain -> DispatcherServlet
       -> handler mapping -> controller -> application service
       -> return value handler -> message converter -> HTTP response
```

Failures can happen before the controller, during binding/validation, inside business logic, or while serializing the response. Diagnose the phase before changing controller code.

Narrow request and response models

JAVA

```java
record CreateOrderRequest(
        @NotBlank String customerId,
        @NotEmpty List<@Valid OrderLineRequest> lines,
        @NotBlank String idempotencyKey) { }

record OrderLineRequest(
        @NotBlank String sku,
        @Positive int quantity) { }

record OrderResponse(UUID id, String status, Instant createdAt) { }
```

Do not bind HTTP JSON directly to a persistence entity. Request DTOs prevent over-posting, define validation, and let API and storage evolve independently.

Controller boundary

JAVA

```java
@RestController
@RequestMapping("/api/orders")
final class OrderController {
    private final OrderService service;

    OrderController(OrderService service) {
        this.service = service;
    }

    @PostMapping
    ResponseEntity<OrderResponse> create(
            @Valid @RequestBody CreateOrderRequest request) {
        OrderResponse created = service.create(request);
        URI location = URI.create("/api/orders/" + created.id());
        return ResponseEntity.created(location).body(created);
    }
}
```

The controller translates HTTP into an application command, invokes one use case, and translates the result. It should not open database transactions manually or contain retry loops.

Structural versus business validation

Bean Validation checks request shape: required text, numeric range, valid nested elements. Business validation may need current state: SKU exists, inventory can be reserved, idempotency key belongs to the same request. Database constraints protect concurrent invariants.

TEXT

```text
JSON syntax -> binding -> structural validation -> authorization
            -> business invariant -> database constraint
```

Do not query the database from a custom field validator by default. It hides I/O and transaction semantics.

Consistent errors

Spring supports RFC-style problem details. Centralize translation:

JAVA

```java
@RestControllerAdvice
final class ApiExceptionHandler {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException exception) {
        ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.NOT_FOUND);
        problem.setTitle("Order not found");
        problem.setDetail(exception.getMessage());
        return problem;
    }
}
```

Do not include stack traces, class names, SQL, credentials, or internal hostnames. Add a stable application error code and trace/correlation identifier where the API contract needs them.

Status decisions

Situation	Typical status
created resource	201
invalid request syntax/shape	400
unauthenticated	401
authenticated but forbidden	403
missing resource	404
idempotency conflict/state conflict	409
semantic validation where contract uses it	422
unexpected server failure	500
temporary dependency unavailable	503

The exact choice is an API contract. Keep it consistent and test response bodies as well as codes.

WATCH OUT | Common mistakes

- Returning 200 for every outcome.
- Catching `Exception` in every controller.
- Leaking entity fields into JSON.
- Treating validation as authorization.
- Returning a raw exception message from a remote system.
- Forgetting validation on nested collection elements.
- Writing an error handler that converts programming defects into 400 responses.

Interview angle

Interviewer: Where do you validate an order request?

Strong answer: MVC binding and Bean Validation reject malformed structure; authorization verifies the caller; the application service enforces state-dependent business invariants; and the database protects concurrent uniqueness. The controller maps failures to a stable problem contract without leaking internals.

Quick check

1. Why use DTOs instead of entities?
2. What is structural validation?
3. Which phase runs before the controller?
4. When is 409 appropriate?
5. What must never appear in an error response?

Practice

- **Foundation:** Build a validated create-order endpoint.
- **Foundation:** Add validation for each nested line.
- **Interview Core:** Define problem responses for four failures.
- **Interview Core:** Test that internal exception text is redacted.
- **SDE-2 Follow-up:** Design idempotent POST behavior under two concurrent requests.

SDE-2 CORE CHAPTER

Chapter 11 - HTTP, JSON, Pagination, Versioning, and Idempotency

An SDE-2 candidate must reason beyond controller annotations. HTTP method semantics, JSON compatibility, pagination stability, caching, and idempotency determine whether clients survive retries and service evolution.

Method semantics

Method	Intent	Safe	Usually idempotent
GET	read representation	yes	yes
POST	create/command	no	no, unless application adds a key
PUT	replace known resource	no	yes
PATCH	partial update	no	contract-dependent
DELETE	remove resource	no	yes in intended final state

Idempotent does not mean the response is identical or that no logs/metrics change. It means repeating the same intended operation has the same externally relevant effect.

JSON compatibility

Adding an optional response field is usually backward-compatible for tolerant clients. Renaming/removing a field, changing number to string, changing nullability, or reinterpreting an enum can break clients.

Use explicit contract models and contract tests. Do not globally change the JSON mapper to fix one endpoint without auditing every consumer.

JAVA

```java
record MoneyResponse(String currency, BigDecimal amount) { }
```

Represent money with a decimal value and currency contract. Avoid binary floating-point for financial amounts.

Content negotiation

Clients express accepted representations through `Accept`; request bodies identify their type with `Content-Type`. A 415 means the request media type is unsupported; a 406 means the server cannot produce an acceptable response.

Do not use URL extensions as the main negotiation strategy in a modern API unless a legacy contract requires it.

Pagination

Offset pagination is simple:

```text
GET /api/orders?page=4&size=50&sort=createdAt,desc
```

but concurrent inserts can shift offsets. Cursor/keyset pagination uses a stable ordered boundary:

```text
GET /api/orders?after=2026-07-30T10:15:00Z,8f2...&limit=50
```

The sort must be deterministic; add a unique tie-breaker. Never expose an unsigned database cursor containing sensitive data. Validate maximum page size.

Idempotent create

```text
client sends Idempotency-Key K + canonical request hash H
        |
        v
atomically claim K
  +-- new -> execute and persist result
  +-- same K/H completed -> return stored result
  +-- same K/different H -> 409 conflict
  +-- same K in progress -> wait/retry contract
```

The database uniqueness constraint is the final concurrency guard. An in-memory map fails across replicas and restarts. Define retention and privacy rules for stored request hashes/results.

API versioning

Version only when compatibility cannot be preserved. Common strategies include URL, header, media type, or platform gateway routing. Spring Framework/Boot versions may add API-versioning support, but the organization still owns lifecycle, deprecation, documentation, observability by version, and client migration.

Avoid versioning every implementation change. Prefer additive response changes and tolerant readers.

WATCH OUT | Common mistakes

- Using POST retries without an idempotency contract.
- Offset pagination without deterministic ordering.
- Returning JPA lazy proxies as JSON.
- Accepting unbounded page sizes.
- Using enum names as eternal public values without evolution planning.
- Treating ETag and cache behavior as an afterthought for hot reads.
- Adding `/v2` but leaving error and authentication semantics undocumented.

Interview angle

Interviewer: A client times out after order creation and retries. How do you avoid duplicates?

Strong answer: The client sends a stable idempotency key; the service atomically records the key with a canonical request fingerprint in durable shared storage; the transaction creates the order and records the result under a uniqueness constraint; same-key/same-request retries return the original outcome, while same-key/different-request returns conflict. I define in-progress and retention behavior.

Quick check

1. Safe versus idempotent?
2. Why can offset pages duplicate items?
3. What makes cursor ordering stable?
4. What must an idempotency record include?
5. When should an API version change?

Practice

- **Foundation:** Map common request failures to HTTP statuses.
- **Interview Core:** Design a stable cursor for `(created_at, id)`.
- **Interview Core:** Define JSON compatibility tests for an evolving response.
- **SDE-2 Follow-up:** Write the state machine for idempotent order creation and recovery after a crash.

SDE-2 CORE CHAPTER

Chapter 12 - Outbound HTTP Clients, Timeouts, and Resilience Boundaries

Calling another service is not a method call with latency. It crosses a network boundary where requests may be delayed, duplicated, partially processed, rejected, or answered after the caller gives up.

Use configured builders

Boot supplies configured builders for supported clients. In a blocking MVC service, `RestClient` is a clear default:

JAVA

```java
@Bean
PaymentClient paymentClient(RestClient.Builder builder,
                            PaymentProperties properties) {
    RestClient client = builder
            .baseUrl(properties.baseUrl().toString())
            .build();
    return new PaymentClient(client);
}
```

JAVA

```java
final class PaymentClient {
    private final RestClient client;

    PaymentClient(RestClient client) {
        this.client = client;
    }

    PaymentResult authorize(PaymentRequest request) {
        return client.post()
                .uri("/authorizations")
                .body(request)
                .retrieve()
                .body(PaymentResult.class);
    }
}
```

Boot 4.1 also supports focused client starters and HTTP service interfaces. A declarative interface reduces repetitive mapping but does not remove timeout, retry, authentication, error, or observability decisions.

Timeout taxonomy

Timeout	Bounds
connection acquisition	waiting for a pool connection
connect	establishing TCP/TLS connection
response/read	waiting for response data
overall request/deadline	total operation budget

An infinite default is not resilience. Choose budgets from the caller's service-level objective and leave time for fallback/error translation.

Retry decision

Retry only when all are true:

1. failure may be transient;
2. operation is idempotent or protected by an idempotency key;
3. enough deadline remains;
4. attempt count/backoff/jitter are bounded;
5. retry load will not amplify an outage.

Do not retry validation errors, authentication failures, or deterministic business rejection. Treat 429/503 according to contract and `Retry-After` where applicable.

Circuit breaker and bulkhead

A circuit breaker prevents repeated calls when failure evidence crosses a threshold; it is not a replacement for timeouts. A bulkhead limits concurrent work so one slow dependency cannot consume every request thread or connection.

Boot does not make an arbitrary resilience library correct automatically. Metrics must distinguish original requests, attempts, timeouts, rejected bulkhead calls, open-circuit calls, and final outcomes.

Error translation

Translate transport details at the adapter boundary:

```text
404 payment account -> domain not-found if contract says so
409 provider conflict -> stable application conflict
429/503 -> temporary dependency failure with retry metadata
malformed response -> integration contract failure
timeout -> outcome unknown, not automatically failed
```

Never return a provider's raw body to public clients.

Transaction interaction

Avoid holding a database transaction while waiting for a remote response. It increases lock and pool duration and still cannot make the remote call atomic with the database commit. Use explicit workflow states, outbox/inbox, compensation, or a saga where needed.

WATCH OUT | Common mistakes

- Creating a new HTTP client per request.
- Setting only a connect timeout.
- Retrying POST without idempotency.
- Retrying at gateway, service, client, and library layers simultaneously.
- Catching every exception and returning an empty success.
- Logging authorization headers or full sensitive bodies.
- Treating a timeout as proof the provider did nothing.

Interview angle

Interviewer: Payment timed out. Should the order retry immediately?

Strong answer: A timeout makes the outcome unknown. I query by the same idempotency key or use a provider status API before creating a second authorization. Any retry is bounded by the end-to-end deadline and protected against duplication. I keep the database transaction short and persist an explicit pending state for recovery.

Quick check

1. Name four timeout boundaries.
2. When is retry safe?
3. What problem does a bulkhead solve?
4. Why is timeout an unknown outcome?
5. Why avoid remote calls inside database transactions?

Practice

- **Foundation:** Configure explicit connect and response timeouts.
- **Interview Core:** Classify HTTP failures as retryable or final.
- **Interview Core:** Design metrics for requests versus attempts.
- **SDE-2 Follow-up:** Build a recovery workflow for unknown payment outcomes.

SDE-2 CORE CHAPTER

Chapter 13 - Data Access, Transactions, and Schema Migrations

Boot can configure a connection pool, JDBC/JPA infrastructure, transaction manager, SQL initialization, and migration tools. It cannot choose correct transaction boundaries or make an unsafe schema rollout compatible.

DataSource auto-configuration

Typical inputs are:

- JDBC API and pool implementation on the classpath;
- a database driver;
- `spring.datasource.*` properties;
- absence of a user-defined `DataSource`.

An embedded database may start locally with no URL. Production must provide an explicit contract. Verify driver, URL, credentials, pool settings, validation, and target database behavior.

PROPERTIES

```
spring.datasource.url=jdbc:mysql://db:3306/orders
spring.datasource.username=orders_app
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=1s
```

Do not copy pool sizes across services. Bound application concurrency and size the pool against database capacity, query time, and transaction duration.

Transaction ownership

JAVA

```java
@Service
final class OrderService {
    @Transactional
    public OrderId create(CreateOrder command) {
        // enforce invariant and persist one unit of work
    }
}
```

Boot contributes the manager; Spring transaction semantics still apply: proxy crossing, default unchecked rollback, propagation, isolation, and thread binding. Revisit SD 04 for those mechanics.

Open-in-view warning

Keeping a persistence context open through web rendering can hide lazy loads in controllers/serialization and create query surprises. Prefer fetching what the use case needs inside the transaction and returning explicit DTOs. Configure and test the selected behavior rather than relying on a default that may change across lines.

Schema migration is deployment coordination

Use Flyway or Liquibase as the canonical migration mechanism. Boot can run migrations during startup, but rollout safety depends on service topology and migration type.

Expand-contract example:

```text
release A: add nullable new column/index without breaking old code
release B: write old and new representation; backfill safely
release C: read new representation after evidence
release D: stop old writes
release E: enforce constraint/drop old column later
```

Large backfills and blocking DDL may not belong in every application instance's startup path. Coordinate one migration job and observe locks, replication, and duration.

Initialization ordering

Do not mix ad hoc `schema.sql`, ORM create/update, and a migration tool without a defined owner. In production, automatic ORM schema mutation is rarely a safe deployment plan.

Failure behavior

- Invalid credentials should fail startup if the database is required.
- A temporary database outage may keep readiness false; liveness should not restart every replica solely because a shared database is down.
- Pool exhaustion requires separating acquisition wait, leaks, slow queries, locks, and transaction duration.
- Migration failure should halt the rollout, not let incompatible code receive traffic.

WATCH OUT | Common mistakes

- Assuming auto-configured H2 proves MySQL behavior.
- Keeping a transaction open across remote calls.
- Returning entities directly from controllers.
- Enabling schema auto-update in production.
- Running a destructive migration before all old instances stop using the field.
- Making every pod race to perform a long backfill.

Interview angle

Interviewer: A deployment starts timing out on connection acquisition. Increase the pool?

Strong answer: First I measure active/idle/pending connections, transaction duration, slow queries, lock waits, remote calls inside transactions, and leak evidence. A larger pool can overload the database. I shorten hold time and bound concurrency, then size the pool from measured database capacity.

Quick check

1. What inputs commonly trigger DataSource auto-configuration?
2. Why can H2 tests mislead?
3. What is expand-contract migration?
4. Why avoid ORM auto-update in production?
5. What does open-in-view conceal?

Practice

- **Foundation:** Trace the beans created for one JDBC configuration.
- **Interview Core:** Design a no-downtime column rename.
- **Interview Core:** Diagnose pool acquisition timeout with five measurements.
- **SDE-2 Follow-up:** Decide whether migrations run in application startup, a deployment job, or both.

SDE-2 CORE CHAPTER

Chapter 14 - Actuator Endpoints, Health, and Operational Access

Actuator exposes management capabilities through technology-neutral endpoints adapted to HTTP or JMX. Adding the starter is only the first step; availability, exposure, access, details, and network reachability are separate decisions.

Add the capability

XML

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

By default, only a limited endpoint set is exposed remotely. Do not expose `env`, `configprops`, `beans`, `heapdump`, `loggers`, or `shutdown` broadly.

Four gates

TEXT

```
endpoint bean exists
      |
      v
endpoint access permits operation
      |
      v
technology exposure includes it
      |
      v
network/security policy permits caller
```

An endpoint removed from the context by access policy is different from an endpoint present but not exposed over HTTP.

Explicit exposure

YAML

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when-authorized
```

Treat the management surface as an administrative API. Use authentication/authorization and network segmentation. A separate management port can isolate traffic, but probes on that port may succeed even when the main server cannot accept connections.

Health contributors

JAVA

```java
@Component
final class PaymentHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        return Health.up()
                .withDetail("mode", "degraded-capable")
                .build();
    }
}
```

This trivial example should not make a live remote call per probe. Health checks need bounded time, safe detail, and clear semantics. A dependency can be degraded without making the process dead.

Health groups

Group indicators by operational decision:

- **liveness:** should the platform restart this process?
- **readiness:** should new traffic be sent here?
- **diagnostic:** what dependencies or subsystems are degraded?

Do not put a shared database or provider in liveness; an outage could restart every replica and amplify failure. Include external dependencies in readiness only after deciding the consequences of removing every instance from service.

Custom endpoints

Use application APIs for business operations. A custom Actuator endpoint is suitable for operator diagnostics or carefully controlled actions:

JAVA

```java
@Endpoint(id = "orderBacklog")
final class OrderBacklogEndpoint {
    @ReadOperation
    BacklogSnapshot snapshot() {
        return new BacklogSnapshot(queuedCount, oldestAge);
    }
}
```

Keep payloads bounded and sanitized. Mutating operations need strict access and audit.

WATCH OUT | Common mistakes

- Exposing every endpoint with *.
- Returning credentials or full configuration in details.
- Calling a slow remote system synchronously on every health request.
- Using health as a high-cardinality diagnostics dump.
- Assuming a separate management port proves the main port works.
- Enabling runtime log-level changes without authorization/audit.

Interview angle

Interviewer: Should database failure make readiness fail?

Strong answer: It depends on whether the instance can serve meaningful traffic without the database and whether all replicas share it. Removing all instances may turn a dependency outage into total unreachability. I keep liveness independent, define a readiness policy from traffic behavior, expose detailed dependency diagnostics separately, and make checks bounded.

Quick check

1. Availability versus exposure versus access?
2. Why is a management port not sufficient security?
3. What operational decision should liveness drive?
4. Why keep health payloads bounded?
5. When is a custom endpoint appropriate?

Practice

- **Foundation:** Expose only health and info.
- **Interview Core:** Design liveness/readiness for a database-backed API.
- **Interview Core:** Threat-model the management surface.
- **SDE-2 Follow-up:** Create a health-group policy for required and optional dependencies.

SDE-2 CORE CHAPTER

Chapter 15 - Metrics, Observations, Traces, and Structured Logging

Observability is evidence for answering a question, not the number of signals emitted. Boot integrates Micrometer metrics and observations, tracing bridges, structured logging, and Actuator endpoints; the application still owns useful names, cardinality, sampling, and privacy.

Signal roles

Signal	Best for
metrics	rates, ratios, saturation, alerts, trends
traces	one request across components/services
logs	discrete events and rich diagnostic context
profiles/dumps	CPU, allocation, threads, memory incidents

Do not use logs as a high-volume metric store or add every business ID as a metric tag.

Low-cardinality metrics

JAVA

```java
@Component
final class OrderMetrics {
    private final Counter accepted;

    OrderMetrics(MeterRegistry registry) {
        this.accepted = Counter.builder("orders.accepted")
                .description("Orders accepted by the application")
                .register(registry);
    }

    void recordAccepted() {
        accepted.increment();
    }
}
```

Good tags have bounded values: operation, outcome, region, dependency. Bad tags are order ID, user ID, raw URL, exception message, or timestamp. Cardinality consumes memory and backend cost.

Observation wraps one operation

An observation can create metrics and tracing spans through configured handlers. Instrument around an application boundary:

JAVA

```java
return Observation.createNotStarted("payment.authorize", registry)
        .lowCardinalityKeyValue("provider", providerName)
        .observe(() -> gateway.authorize(command));
```

Never put credentials, card data, request bodies, or unbounded identifiers into observation tags.

Trace propagation

Trace context must cross supported HTTP or messaging instrumentation. New unmanaged threads, manual executors, and asynchronous boundaries can lose it. Verify propagation with an integration test and logs containing trace identifiers.

Sampling means not every request has a complete trace. Alerts and correctness must not depend on trace presence.

Structured logs

TEXT

```
timestamp level service environment event outcome trace_id message
```

Use stable event names and fields. Avoid concatenated prose as the only representation. Log one failure at the layer that owns the response; repeated stack traces at every layer increase noise.

Redact authentication headers, tokens, cookies, secrets, personal data, and sensitive payloads. Hashing an identifier can still be personal data if it remains linkable.

Service-level evidence

For an API, start with:

- request rate by normalized route and outcome;
- latency distribution, not only averages;
- error ratio by controlled category;
- executor and connection-pool saturation;
- dependency attempts, timeouts, and circuit state;
- readiness state and restart count;
- business acceptance/rejection counts with bounded dimensions.

WATCH OUT | Common mistakes

- Tagging metrics with raw path IDs.
- Alerting on a single instance rather than user impact.
- Logging the same exception five times.
- Assuming trace context crosses every executor automatically.
- Recording high-cardinality exception messages.
- Publishing `prometheus` or log-management endpoints publicly.

Interview angle

Interviewer: Latency rose. Which signal do you inspect first?

Strong answer: I confirm user-impacting latency by normalized route and percentile, then correlate saturation and dependency timing. Traces identify where representative requests spent time; logs explain discrete failures. I compare request rate, error rate, pool/executor queues, database latency, and deployment changes before attributing cause.

Quick check

1. Why are averages weak for tail latency?
2. What is metric cardinality?
3. What does an observation produce?
4. Why can trace sampling not prove absence?
5. Which data must be redacted?

Practice

- **Foundation:** Define four low-cardinality tags.
- **Interview Core:** Repair a metric tagged by customer ID.
- **Interview Core:** Correlate a timeout across metric, trace, and log evidence.
- **SDE-2 Follow-up:** Design alerts for latency, errors, and saturation without paging on harmless noise.

SDE-2 CORE CHAPTER

Chapter 16 - Availability, Probes, and Graceful Shutdown

Process running, application live, application ready, and dependency healthy are different statements. Platforms act on probes, so incorrect semantics can turn one dependency incident into a restart storm or total traffic outage.

Availability states

TEXT

```
startup:
  liveness BROKEN -> CORRECT
  readiness REFUSING_TRAFFIC -> ACCEPTING_TRAFFIC

shutdown:
  readiness ACCEPTING_TRAFFIC -> REFUSING_TRAFFIC
  stop accepting new requests
  drain in-flight work within timeout
  close context and resources
```

Boot exposes Kubernetes-oriented health groups at:

TEXT

```
/actuator/health/liveness
/actuator/health/readiness
```

Additional main-port paths can reduce false confidence when management uses a separate port.

Liveness rule

Liveness asks whether restarting this process is likely to repair it. It should normally exclude shared external systems. If a database fails and every replica reports dead, the platform restarts all of them while the database is still unavailable.

Readiness rule

Readiness asks whether this instance should receive new traffic. Include only dependencies required for the served traffic and consider service-wide impact. An optional recommendation provider should degrade a feature, not remove the order API from service.

Startup probe

A startup probe can protect a legitimately slow application from liveness kills. It should not normalize unbounded migrations or cache warmups. Measure startup distribution and make required startup work finite.

Graceful shutdown

PROPERTIES

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
```

During graceful shutdown the server rejects new requests and permits in-flight work to finish within the phase timeout. The orchestrator must provide a termination grace period longer than application drain plus safety margin.

Shutdown also requires:

- stop message intake or scheduler claims;
- stop accepting new application tasks;
- await bounded executors;
- release database and HTTP pools;
- complete or recover durable work;
- preserve termination reason and metrics.

Long requests and async work

A 20-second shutdown budget cannot safely drain a 2-minute request. Bound request deadlines. For durable background work, lease/claim units so another instance can recover after lease expiry. In-memory queued work can be lost on termination.

Deployment timeline

```text
new instance starts -> readiness passes -> receives traffic
old instance marked unready -> load balancer converges
preStop/drain window -> shutdown signal -> in-flight completion
force kill only after grace period
```

Account for load-balancer propagation; readiness changing in-process is not instantly observed everywhere.

WATCH OUT | Common mistakes

- Using the same check for liveness and readiness.
- Including every shared dependency in liveness.
- Setting shutdown timeout longer than platform grace.
- Starting a long migration in every replica.
- Keeping an unbounded task executor.
- Assuming an in-memory queue survives rolling deployment.
- Returning ready before mandatory startup state is safe.

Interview angle

Interviewer: Deployments drop requests despite graceful shutdown. Why?

Strong answer: I compare readiness transition, endpoint removal and load-balancer convergence, pre-stop delay, server drain, application request deadlines, executor work, and platform termination grace. Graceful shutdown cannot save requests still routed after the instance begins closing or work longer than the allowed budget.

Quick check

1. What action does liveness trigger?
2. What action does readiness trigger?
3. When is a startup probe appropriate?
4. What must platform grace exceed?
5. How should durable background work recover?

Practice

- **Foundation:** Classify five checks as live, ready, or diagnostic.
- **Interview Core:** Design probes for required database and optional cache.
- **Interview Core:** Calculate a shutdown budget from request deadlines.
- **SDE-2 Follow-up:** Diagnose a zero-downtime rollout that still returns connection resets.

SDE-2 CORE CHAPTER

Chapter 17 - Testing Ladder, Contexts, Slices, and Testcontainers

The strongest Boot test suite uses the smallest boundary that proves each claim. Full-context tests are valuable, but they are expensive and often conceal which behavior failed.

The ladder

TEXT

```
pure unit
  -> configuration binding / ApplicationContextRunner
  -> web or data slice
  -> full application context without server
  -> random-port process integration
  -> target dependency with Testcontainers
  -> deployment smoke / end-to-end
```

Each level adds realism and failure surface. Do not ask one level to prove what it cannot observe.

Pure unit test

JAVA

```java
@Test
void rejectsEmptyOrder() {
    OrderService service = new OrderService(repository, paymentGateway);
    assertThatThrownBy(() -> service.create(emptyCommand()))
            .isInstanceOf(InvalidOrderException.class);
}
```

No Boot context is needed for domain behavior constructed from plain Java dependencies.

Auto-configuration test

`ApplicationContextRunner` starts a focused context and exposes rich assertions:

JAVA

```java
runner.withPropertyValues("acme.audit.enabled=false")
      .run(context -> assertThat(context)
              .doesNotHaveBean(AuditSink.class));
```

This is the right tool for condition, back-off, and binding matrices.

Test slices

A web slice loads controller-oriented infrastructure while replacing/importing collaborators deliberately. Data slices load data infrastructure for the selected technology. Boot 4 uses more focused test modules/starters, but the principle is unchanged: a slice is a restricted context, not a smaller production process.

JAVA

```java
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;
}
```

Test request binding, validation, status, JSON, and error translation. Do not mock MVC itself.

Full context

JAVA

```java
@SpringBootTest
class ApplicationWiringTest {
    @Test
    void contextStarts() { }
}
```

A single context-start test detects broken assembly but has weak diagnostics alone. Add assertions about important beans, properties, and conditions.

Random-port tests start the real embedded server and exercise serialization, filters, server configuration, and network behavior. They are slower and require reliable port/client cleanup.

Testcontainers

Use the target database or broker when dialect, locking, migrations, transaction isolation, or driver behavior matters. Boot service connections can derive connection details from supported containers, reducing manual dynamic properties.

Still pin container versions, wait for readiness, isolate data, and record logs on failure. A container test is not production load testing.

Context cache and test pollution

Spring caches compatible contexts. Excessive distinct property sets, profiles, mocks, and dirty contexts slow the suite. Mutable singleton state and leaked threads make order-dependent tests.

Measure context count and move behavior down the ladder. Use `@DirtiesContext` only when the test genuinely invalidates the context.

Commit behavior

Rollback-by-default tests may not prove commit-time constraints, after-commit events, or behavior from a new transaction. Add tests that commit and verify from a separate transaction where durability matters.

WATCH OUT | Common mistakes

- Using `@SpringBootTest` for every service method.
- Mocking repositories in a test claimed to prove SQL.
- Using H2 for MySQL-specific locking behavior.
- Sharing mutable container data across tests.
- Assuming context-start proves endpoint correctness.
- Making tests depend on execution order.
- Testing only rollback paths and never commit evidence.

Interview angle

Interviewer: How do you choose a Boot test annotation?

Strong answer: I start from the claim. Pure logic gets a unit test; condition/binding behavior gets `ApplicationContextRunner`; MVC mapping gets a web slice; cross-bean assembly gets a full context; real server filters/serialization get random port; database-specific behavior gets a pinned target container. I minimize context variants and add commit evidence where required.

Quick check

1. What does a web slice intentionally omit?
2. When is `ApplicationContextRunner` best?
3. What extra behavior does random port prove?
4. Why can rollback tests mislead?
5. What causes context-cache fragmentation?

Practice

- **Foundation:** Move one service test from full context to pure Java.
- **Interview Core:** Write a four-case auto-configuration matrix.
- **Interview Core:** Test validation/error JSON with a web slice.
- **SDE-2 Follow-up:** Design the minimum test portfolio for an idempotent database-backed POST.

SDE-2 CORE CHAPTER

Chapter 18 - Packaging, Layers, Containers, and Runtime Resources

The build is not complete when tests pass. The deployed artifact must contain the intended classes and dependencies, start with the supported command, expose the right ports, run as a restricted user, respect resource limits, and stop within the platform budget.

Executable jar anatomy

The Boot plugin repackages application classes and dependencies with launcher metadata. Verify:

BASH

```
./mvnw clean verify
jar tf target/order-service.jar | head
java -jar target/order-service.jar --spring.profiles.active=smoke
```

Do not deploy an IDE classpath. Test the exact artifact produced by CI.

Layered jars

Application code changes more often than dependencies. Layer metadata allows container builds to cache stable dependency layers separately from snapshot dependencies and application classes.

TEXT

```
dependencies
spring-boot-loader
snapshot-dependencies
application
```

Layering improves rebuild/pull efficiency. It does not reduce runtime memory by itself.

Buildpacks

BASH

```
./mvnw spring-boot:build-image \
  -Dspring-boot.build-image.imageName=registry.example/orders:1.4.2
```

Cloud Native Buildpacks create an OCI image without a handwritten Dockerfile. They choose build/run images, JVM settings, layers, and metadata. Pin trusted builders/run images according to organizational policy and scan the final image.

A deliberate Dockerfile path

When requirements need a custom image, use a multi-stage build and a minimal runtime image. Copy the verified jar, run as non-root, set an explicit entry point, and avoid shell surprises.

DOCKERFILE

```
FROM eclipse-temurin:21-jre
RUN useradd --system --uid 10001 appuser
WORKDIR /app
COPY order-service.jar app.jar
USER 10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

The image tag should be immutable or paired with a digest. Never bake environment secrets into image layers.

Resource awareness

Container memory includes heap, metaspace, code cache, thread stacks, direct buffers, native libraries, and side effects such as memory-mapped files. Setting heap equal to the container limit invites termination.

Measure:

- live heap after warmup;
- allocation and GC pause behavior;
- thread count and stack size;
- direct/network buffer usage;
- class/metaspace growth;
- headroom during traffic spikes and diagnostics.

CPU limits can change available processors and default pool sizing. Do not let every framework/library derive a large pool independently.

Supply-chain evidence

Produce and retain:

- dependency graph and lock/verification metadata;
- software bill of materials;
- vulnerability results with reachability/context;
- artifact checksum and signature where used;
- base-image provenance;
- build and deployment version information.

WATCH OUT | Common mistakes

- Running as root.
- Using `latest` as the only deployable identity.
- Copying the Maven cache and source tree into the runtime image.
- Setting heap to 100 percent of container memory.
- Expecting a Docker `HEALTHCHECK` to replace orchestrator probes.
- Installing troubleshooting tools in every production image without risk review.
- Building one artifact locally and deploying another from CI.

Interview angle

Interviewer: A pod with a 1 GiB limit is OOM-killed although `-Xmx768m`. Explain.

Strong answer: The limit covers more than heap: metaspace, code cache, thread stacks, direct buffers, native libraries, and process overhead. I inspect container termination reason and native memory evidence, count threads/direct buffers, leave measured headroom, and tune workload/pools before simply lowering or raising heap.

Quick check

1. What does executable repackaging add?
2. Why use layers?
3. What does buildpack generation still require from the team?
4. Which memory areas exist outside heap?
5. Why use immutable image identity?

Practice

- **Foundation:** Run the exact packaged jar in CI.
- **Interview Core:** Compare a buildpack image and custom Dockerfile.
- **Interview Core:** Create a memory budget for a 1 GiB container.
- **SDE-2 Follow-up:** Design artifact provenance from source commit to deployed digest.

SDE-2 CORE CHAPTER

Chapter 19 - Startup Performance, AOT, Native Images, and Upgrades

Optimize from measured constraints. A service that starts in four seconds and deploys safely does not need native-image complexity because another team published a benchmark.

Measure the phase

Startup time can be dominated by:

- classpath scanning and configuration parsing;
- bean construction and post-processing;
- database migrations;
- remote calls in initialization;
- cache warmup/data loading;
- embedded server startup;
- JIT and first-request warmup.

Use structured startup steps, Java Flight Recorder, profiles, condition reports, and bean counts. Separate process start, context refresh, ready event, and first useful response.

Safe reductions

1. Remove unused starters and transitive capabilities.
2. Eliminate remote calls from constructors and post-processors.
3. Bound or defer optional warmup.
4. Narrow component/entity/repository scans.
5. Simplify excessive context customizers.
6. Make migrations a deliberate deployment step.

Global lazy initialization can improve startup but moves errors and latency to the first request. Use it only with warmup and failure evidence.

AOT processing

Ahead-of-time processing analyzes application structure during the build and generates code/hints that reduce some runtime discovery. It changes build behavior and may reveal dynamic patterns that need explicit hints.

AOT is useful for JVM deployment too; native compilation is a separate choice. Test AOT output in CI, not only ordinary JVM mode.

Native image trade-offs

Potential benefits:

- fast process startup;
- lower steady memory for some workloads;
- good fit for scale-to-zero or short-lived processes.

Costs:

- longer/more complex builds;
- closed-world constraints for reflection, resources, serialization, and dynamic proxies;
- different profiling and peak-throughput behavior;
- dependency compatibility work.

Benchmark the actual service, traffic, container limits, and deployment model.

Upgrade discipline

For a maintenance upgrade:

1. read Boot and dependency release notes;
2. remove expired overrides;
3. compare dependency trees and configuration changelog;
4. compile with warnings treated seriously;
5. run behavior, migration, packaging, startup, and smoke tests;
6. canary and observe before full rollout.

For major upgrades, move through the recommended latest maintenance line first. Boot 4 introduced modularized starters and newer platform baselines; code that depended on internal auto-configuration classes or deprecated starters needs explicit migration.

Configuration drift

Unknown, renamed, or removed properties can silently stop affecting behavior. Keep configuration metadata checks, compare effective values, and inspect deprecation logs. A successful context does not prove every old property still works.

WATCH OUT | Common mistakes

- Enabling lazy initialization to hide cycles or invalid beans.
- Timing only `main` to ready and ignoring first-request warmup.
- Selecting native image without workload evidence.
- Treating internal auto-configuration classes as stable API.
- Upgrading Boot while pinning an incompatible Framework version.
- Ignoring removed configuration properties because tests still start.
- Combining framework upgrade, database migration, and major refactor in one rollout.

Interview angle

Interviewer: How would you reduce a 90-second startup?

Strong answer: I decompose startup steps, separate context work from migrations/remote warmup, identify the longest bounded operations, remove unused capabilities and constructor I/O, and decide what is mandatory before readiness. I validate improvements in the packaged deployment. AOT/native or lazy initialization comes only after simpler causes are measured.

Quick check

1. Which timestamps define startup?
2. What risk does global lazy initialization add?
3. Is AOT identical to native compilation?
4. When can native images help?
5. Why compare configuration across upgrades?

Practice

- **Foundation:** Record process, context, ready, and first-response times.
- **Interview Core:** Remove one unused starter and compare startup evidence.
- **Interview Core:** Write a major-upgrade verification matrix.
- **SDE-2 Follow-up:** Decide JVM versus native for a scale-to-zero API using measured assumptions.

SDE-2 CORE CHAPTER

Chapter 20 - Security Configuration and Production Architecture

Spring Security deserves its own book. This chapter defines the Boot-level responsibilities every backend engineer must understand: secure defaults, management-surface protection, forwarded headers, CORS/CSRF boundaries, secrets, dependency risk, and production configuration.

Adding Security changes behavior

When Security is on the classpath, Boot can register a default security setup only while the application has not supplied its own relevant beans. Defining a `SecurityFilterChain` causes parts of that default to back off, so the application must then authorize the complete surface deliberately.

JAVA

```java
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
    return http
            .authorizeHttpRequests(requests -> requests
                    .requestMatchers("/actuator/health/**").permitAll()
                    .requestMatchers("/actuator/**").hasRole("OPS")
                    .anyRequest().authenticated())
            .build();
}
```

This is illustrative, not a complete authentication design. Browser sessions, token APIs, OAuth2 resource servers, and internal mTLS have different CSRF and credential rules.

CORS is browser permission

CORS tells a browser whether frontend origin A may call API origin B. It does not authenticate the caller and does not stop server-to-server requests.

Avoid * with credentials. Enumerate trusted origins/methods/headers and test preflight behavior through the security filter chain.

CSRF depends on credential transport

Cookie-based browser authentication is automatically attached by the browser and needs CSRF protection. A stateless API using an authorization header has a different threat model. Do not disable CSRF merely because an endpoint returns JSON.

Proxy and forwarded headers

Behind a trusted proxy, the application may need original scheme, host, port, and client information. Trust forwarded headers only from controlled proxies; otherwise a client can spoof secure links, redirect targets, rate-limit identity, or audit data.

Define:

- which proxy terminates TLS;
- which headers it overwrites;
- which network may reach the application directly;
- how generated absolute URLs are tested.

Secrets and diagnostics

Sensitive values can leak through startup errors, environment endpoints, configuration dumps, request logging, heap dumps, thread names, metrics tags, and support bundles. Protect diagnostic endpoints and establish redaction before incidents.

Production architecture checklist

- separate business API and operator permissions;
- authentication and authorization tested for allow and deny paths;
- TLS and proxy trust documented;
- request/body limits and timeouts configured;
- actuator exposure allowlisted;
- secrets externally delivered and rotated;
- dependency/image scanning tied to patch ownership;
- audit events durable where required;
- security headers and CORS behavior verified at the edge;
- error responses reveal no internals.

WATCH OUT | Common mistakes

- Assuming Boot secures custom Actuator exposure automatically.
- Permitting health with a broad matcher that also exposes details.
- Disabling CSRF without identifying credential transport.
- Using CORS as authentication.
- Trusting client-supplied forwarded headers directly.
- Printing configuration during support incidents.
- Hard-coding secrets in `application-prod.yml`.

Interview angle

Interviewer: You added a custom filter chain and Actuator became public. Why?

Strong answer: Security auto-configuration backed off when the application supplied its own chain, and the custom rules did not cover the management surface correctly. I define explicit matchers and order for operator versus application endpoints, test unauthorized/authorized paths, restrict network exposure, and verify health detail policy.

Quick check

1. What can cause security auto-configuration to back off?
2. What does CORS protect?
3. When is CSRF relevant?
4. Why are forwarded headers a trust decision?
5. Which diagnostics may contain secrets?

Practice

- **Foundation:** Separate public readiness from private health details.
- **Interview Core:** Threat-model CORS and CSRF for cookie and token clients.
- **Interview Core:** Test a management endpoint allow/deny matrix.
- **SDE-2 Follow-up:** Design trusted-proxy handling for two load-balancer layers.

SDE-2 CORE CHAPTER

Chapter 21 - Production Incidents and Diagnostic Playbooks

Strong incident response moves from symptom to evidence to the smallest safe change. Boot provides condition reports, failure analyzers, Actuator endpoints, metrics, thread/heap dumps, startup steps, and configuration origins; use each within its security boundary.

Universal diagnostic loop

TEXT

```
1. Define user impact and start time.
2. Compare recent deploy/config/dependency/platform changes.
3. Classify startup, request, dependency, resource, or shutdown phase.
4. Gather bounded evidence before restarting or changing flags.
5. Form competing hypotheses.
6. Run the cheapest discriminating check.
7. Mitigate user impact.
8. Correct and add a regression signal.
```

Startup: missing bean

Evidence order:

1. full failure analyzer message and cause chain;
2. primary configuration and scan/import boundary;
3. active profiles and property origins;
4. positive/negative condition matches;
5. dependency graph differences;
6. user bean type/qualifier and back-off rules.

Do not add component scanning to the company root as a quick fix.

Startup: configuration binding

Record canonical property, supplied name, source origin, raw format without secret value, target type, conversion, and validation failure. Check renamed/removed properties after upgrades. Fix ownership rather than adding a silent default.

Runtime: sudden 404

Separate:

- route not registered (`mappings` evidence);
- wrong context path/base path;
- proxy/gateway rewrite;
- security hiding or rejecting path;
- wrong HTTP method/content negotiation;
- instance running a different version.

Runtime: latency and pool exhaustion

Correlate request percentiles, active requests, executor queues, database-pool pending count, transaction duration, query/lock time, outbound client pools, retry attempts, GC, and CPU throttling. Increasing all pools can move overload downstream.

Runtime: memory termination

Distinguish Java `OutOfMemoryError`, container OOM kill, native allocation failure, and deliberate platform eviction. Capture heap/native/thread evidence when safe. Compare heap, thread count, direct buffers, class growth, request sizes, caches, and recent feature toggles.

Readiness flapping

Check which health contributor changes status, its timeout and cache, whether every instance shares the dependency, and what the platform does when all become unready. A noisy optional dependency should not repeatedly remove healthy core capacity.

Deployment: requests dropped on shutdown

Compare readiness refusal time, load-balancer convergence, `preStop`, termination signal, grace period, server drain timeout, request deadlines, async executors, and message-consumer behavior. Reproduce with a slow but bounded request during rollout.

Safe operational endpoints

- `health`: state and groups;
- `metrics`/Prometheus: rates, latency, saturation;
- `conditions`: auto-configuration decisions;
- `configprops`/`env`: origins and sanitized configuration;
- `mappings`: registered routes;
- `threaddump`/`heapdump`: high-sensitivity diagnostics;
- `startup`: recorded startup steps.

Expose only what operators need, authorize access, and audit high-risk retrieval.

WATCH OUT | Common mistakes

- Restarting before collecting evidence.
- Enabling debug logging globally during peak load.
- Printing secrets to compare configuration.
- Increasing readiness timeouts without finding the slow phase.
- Treating one stack trace as the root cause.
- Making three configuration changes in one mitigation.
- Keeping temporary diagnostic exposure after the incident.

Interview angle

Interviewer: Local succeeds; production fails during context startup. What is your sequence?

Strong answer: I compare artifact checksum, Java/Boot version, resolved dependencies, launch arguments, profiles and property origins, then inspect the failure analyzer and condition report. I reproduce with the production-like packaged artifact and sanitized configuration contract. I do not assume environment difference means "network" or expose all config.

Quick check

1. What is the first incident question?
2. Which endpoint proves route registration?
3. Why can larger pools worsen latency?
4. Container OOM kill versus Java OOME?
5. What evidence is needed for readiness flapping?

Practice

- **Foundation:** Classify ten symptoms by lifecycle phase.
- **Interview Core:** Write a missing-bean decision tree.
- **Interview Core:** Diagnose readiness flapping without restarts.
- **SDE-2 Follow-up:** Lead a latency incident with three competing hypotheses and discriminating checks.

SDE-2 CORE CHAPTER

Chapter 22 - Realistic Spring Boot Interview Rounds

Answer each prompt aloud before reading the model response. A strong answer identifies dependency, condition, property origin, application/context boundary, runtime failure semantics, and evidence.

Answer control loop

```text
1. Clarify the user-visible invariant and environment.
2. Name the Boot capability and underlying Spring mechanism.
3. Trace dependency -> condition -> bean -> property origin.
4. Mark HTTP/thread/transaction/process/deployment boundaries.
5. Predict normal, failure, retry, and shutdown behavior.
6. Select the smallest safe configuration or design.
7. Prove it with focused tests and operational signals.
```

1. Framework versus Boot

Interviewer: What exactly does Boot add?

Strong answer: Framework supplies the container, AOP, transactions, MVC, validation, and testing foundations. Boot adds application bootstrap, conditional auto-configuration, curated dependencies/starters, external-configuration conventions, executable packaging, Actuator integration, and test support. Boot contributes ordinary Spring definitions based on evidence; it does not replace Framework behavior.

2. Starter versus auto-configuration

Interviewer: Is a starter an auto-configuration?

Strong answer: No. A starter is a dependency descriptor. Its libraries may place classes and auto-configuration modules on the classpath. Auto-configuration is runtime configuration selected by conditions. The BOM manages versions and the build plugin packages/runs; those are separate responsibilities.

3. @SpringBootApplication

Interviewer: Explain it and one placement failure.

Strong answer: It identifies Boot configuration, enables auto-configuration, and component-scans from its package. Putting it in a leaf package can exclude components; putting it too high can scan unintended code. I keep it at the application root and use explicit imports for module boundaries.

4. Startup sequence

Interviewer: What happens after `SpringApplication.run`?

Strong answer: Boot prepares listeners/initializers and the environment, determines application type, creates and loads the context, applies user and conditional auto-configuration, refreshes beans, starts the embedded server if needed, runs startup runners, then publishes readiness. I classify a failure by the last completed phase.

5. Auto-configuration back-off

Interviewer: Why did defining one bean change startup?

Strong answer: A missing-bean condition in an auto-configuration no longer matched, so Boot withheld its default. Back-off is local to the condition; related infrastructure may remain. I inspect the condition report and public bean types rather than excluding the whole subsystem.

6. Local DataSource only

Interviewer: Local creates a DataSource, CI does not.

Strong answer: An embedded driver may be present locally or test scope may differ. I compare resolved classpaths, profiles, external URL/driver configuration, user beans, and condition report. I make the target dependency explicit and test the intended database contract.

7. Property precedence

Interviewer: YAML says 8080, process listens on 9090.

Strong answer: Command-line, system, environment, external/profile-specific configuration, or test overrides may have higher precedence. I inspect the winning property source and origin, active profiles, imports, and relaxed binding. Packaged YAML is not the universal winner.

8. Typed configuration

Interviewer: Why not ten `@Value` fields?

Strong answer: A cohesive `@ConfigurationProperties` type gives one namespace, typed conversion, explicit units, metadata, validation, immutable construction, and focused binding tests. I keep secrets redacted and model refresh separately if values rotate.

9. Profiles

Interviewer: Would you create profiles for every customer?

Strong answer: No. Profiles select coherent configuration groups, not tenant data or unlimited feature combinations. Customer values belong in data or typed external configuration; feature delivery needs a governed feature-flag mechanism. Profile explosion makes effective behavior hard to predict.

10. Custom starter

Interviewer: When is a company starter justified?

Strong answer: When many applications need one stable integration contract and defaults. I separate core API from Boot assembly, register auto-configuration through imports metadata, back off for user beans, validate typed properties, test the condition matrix, and publish version compatibility/migration guidance.

11. Controller DTOs

Interviewer: Why not expose the entity?

Strong answer: It couples API and persistence, risks over-posting and lazy loads, and makes compatibility difficult. I use request/response DTOs, perform structural validation at binding, enforce state-dependent invariants in the application service, and preserve concurrent invariants in the database.

12. Error handling

Interviewer: How do you build consistent errors?

Strong answer: A central advice translates known application exceptions to stable problem details and codes. Unknown defects stay 500 and are logged once with correlation. I never expose stack traces, SQL, credentials, or provider bodies, and I contract-test both status and shape.

13. Duplicate POST

Interviewer: A client retries after timeout.

Strong answer: The client supplies an idempotency key. The service atomically claims it with a canonical request fingerprint and persists outcome under a uniqueness constraint. Same key and request returns the original result; same key and different request conflicts. In-progress/crash recovery and retention are explicit.

14. Outbound timeout

Interviewer: Is a timeout a failure?

Strong answer: It is a caller-side deadline; the remote outcome may be unknown. I query/reconcile using the same idempotency key before repeating a non-idempotent operation. Client pools, connect/read/overall timeouts, retry budget, and observability are explicit.

15. Retry policy

Interviewer: Retry every 5xx three times?

Strong answer: No. I retry only transient classifications, with an idempotent operation/key, remaining end-to-end deadline, bounded exponential backoff and jitter, and protection against layered amplification. I respect 429/`Retry-After` contracts and expose attempts separately from requests.

16. Actuator exposure

Interviewer: Why is adding Actuator not enough?

Strong answer: Endpoint existence, access policy, HTTP/JMX exposure, network reachability, and authorization are separate gates. I allowlist endpoints, restrict management traffic, sanitize values, and test access. Heap/env/loggers/shutdown are especially sensitive.

17. Liveness versus readiness

Interviewer: Database is down. Should liveness fail?

Strong answer: Normally no; restarting every process cannot repair a shared database and may amplify load. Readiness depends on whether the instance can serve useful traffic and the effect of removing all replicas. I keep dependency diagnostics separate and checks bounded.

18. Graceful shutdown

Interviewer: Why are requests still dropped?

Strong answer: I trace readiness refusal, load-balancer convergence, pre-stop delay, server drain, request deadlines, executor/message work, and platform grace. Application shutdown timeout must fit inside platform grace, and durable work needs lease/recovery rather than an in-memory queue.

19. Test choice

Interviewer: When use `@SpringBootTest`?

Strong answer: For cross-bean application assembly or a real server boundary, not every method. Pure logic stays unit-tested; conditions/binding use `ApplicationContextRunner`; controllers use a web slice; database-specific behavior uses the target container. The claim selects the boundary.

20. Context tests are slow

Interviewer: How do you reduce suite time?

Strong answer: Count distinct contexts and identify property/profile/mock variations. Move logic down to unit tests, use focused slices/runners, standardize shared context configuration, remove unnecessary `@DirtiesContext`, and fix leaked mutable state/threads. Parallelism comes after isolation.

21. Migration safety

Interviewer: Rename a heavily used column without downtime.

Strong answer: Expand with the new representation, deploy code compatible with both, dual-write/backfill with bounded load and evidence, switch reads, stop old writes after old instances are gone, then contract/drop later. I coordinate DDL/locks and do not run a destructive change in every pod startup.

22. Pool exhaustion

Interviewer: Increase Hikari pool size?

Strong answer: First separate acquisition wait, active/idle/pending counts, transaction duration, slow queries, lock waits, remote calls inside transactions, and leaks. A larger pool can overload the database. I shorten hold time and bound concurrency, then size from database capacity.

23. OOM-killed container

Interviewer: Heap is below the limit, so why killed?

Strong answer: Container memory includes heap, metaspace, code cache, thread stacks, direct buffers, native libraries, and overhead. I distinguish Java OOME from platform kill, inspect native/thread evidence, and leave measured headroom rather than setting heap to the limit.

24. Startup regression

Interviewer: Startup grew from 8 to 60 seconds.

Strong answer: I compare build/dependency/config changes and structured startup steps, isolate context work, migrations, constructor I/O, cache warmup, and server time, then remove or defer the dominant optional work. Lazy/AOT/native comes only after the phase is measured.

25. Boot upgrade

Interviewer: How do you upgrade a major Boot line?

Strong answer: Move to the latest maintenance release of the current line, remove deprecations/overrides, read migration and dependency/configuration changelogs, compare resolved graphs, run compilation, behavior, target-database, packaging, startup, security, and smoke tests, then canary with rollback. I do not pin an incompatible Framework version.

26. Production 404 after deployment

Interviewer: Code contains the mapping, but clients get 404.

Strong answer: I verify artifact version, `mappings`, application/context path, gateway rewrite, HTTP method, security behavior, and instance routing. Source code presence does not prove registration or that traffic reached the expected instance.

27. Readiness flapping

Interviewer: Pods repeatedly enter and leave service.

Strong answer: I identify the exact health contributor, timeout, status aggregation, dependency sharing, and platform thresholds; correlate dependency latency and probe traffic; then choose whether the dependency belongs in readiness. I add hysteresis/cache only with clear stale-state bounds.

28. Observability design

Interviewer: What do you instrument first?

Strong answer: User-facing request rate, latency distribution, errors, and saturation; then dependency attempts/timeouts and core business outcomes with bounded dimensions. Traces explain paths and logs explain events. I ban unbounded IDs and secrets from tags.

Readiness signal

You are ready to use Spring Data and specialist Spring modules when answers begin with dependency, condition, origin, boundary, and evidence rather than a list of annotations.

SDE-2 CORE CHAPTER

Chapter 23 - Practice, Solutions, Readiness, and Sources

Complete each task before reading its solution sketch. The goal is to predict runtime behavior, produce evidence, and communicate trade-offs.

Cumulative assessment 1 - Application assembly

1. **Foundation:** Build and package a one-endpoint MVC service.
2. **Foundation:** Draw the startup phases through readiness.
3. **Interview Core:** Explain starter, BOM, plugin, and auto-configuration separately.
4. **Interview Core:** Diagnose a component outside the scan root.
5. **SDE-2 Follow-up:** Review a pull request that adds six unrelated starters.

Solution sketch

Prove the packaged jar starts and serves the endpoint. The application root defines discovery. A starter changes the classpath; the BOM aligns versions; the plugin packages; auto-configuration registers conditional definitions. Unused starters expand attack, startup, and behavior surface and need removal or explicit justification.

Cumulative assessment 2 - Conditions and configuration

1. Predict a missing-bean back-off result.
2. Identify the winner among packaged YAML, external profile YAML, environment, and CLI.
3. Convert scattered timeout strings into typed properties.
4. Debug a critical optional config import.
5. Design a six-case custom auto-configuration test matrix.

Solution sketch

User definitions register before auto-configuration, so a matching user bean makes the default back off. Determine values from actual ordered property sources and origins. Use typed `Duration`/`DataSize` with explicit units and validation. Required imports must fail fast. Test default, disabled, missing prerequisite, user override, invalid binding, and applicable/non-applicable application type.

Cumulative assessment 3 - HTTP and dependencies

1. Define structural and business validation for order creation.
2. Design one stable problem response.
3. Create a `(createdAt,id)` cursor.
4. Specify idempotency-key states.
5. Allocate a 2-second request deadline across database and payment calls.

Solution sketch

Bind to a DTO, structurally validate it, authorize, enforce state in the service, and use a database constraint for concurrency. Error shape includes stable code/status/title/correlation without internals. Cursor ordering includes a unique tie-breaker. Idempotency distinguishes new/in-progress/completed/conflicting fingerprints. Timeout allocation reserves response/error budget and never lets nested retries exceed the caller deadline.

Cumulative assessment 4 - Operations and testing

1. Separate liveness, readiness, and diagnostics for three dependencies.
2. Allowlist Actuator endpoints and roles.
3. Define four low-cardinality metrics.
4. Select tests for domain, binding, controller, database locking, and packaged server.
5. Diagnose a readiness-flapping deployment.

Solution sketch

Liveness describes process repairability; readiness controls traffic; detailed dependency health remains diagnostic. Expose only required operator endpoints. Metrics use controlled route/outcome/dependency dimensions. Use unit, context runner, web slice, target container, and random-port/smoke respectively. For flapping, identify the contributor and timeout, correlate dependency evidence, and decide whether it belongs in readiness before tuning thresholds.

Cumulative assessment 5 - Delivery and incidents

1. Explain a layered executable jar.
2. Budget heap and non-heap memory for a 1 GiB container.
3. Design expand-contract migration for a column rename.
4. Investigate a 50-second startup regression.
5. Lead a pool-exhaustion incident.

Solution sketch

Layers separate stable dependencies from changing application code for image reuse. Leave measured room for metaspace, stacks, direct/native memory, and spikes. Expand, dual-write/backfill, switch reads, stop old writes, then contract after compatibility evidence. Decompose startup steps. For pools, measure acquisition, active/pending, transaction/query/lock duration, remote calls, and leaks before resizing.

Predict the outcome

1. A user bean matches `@ConditionalOnMissingBean`: default backs off.
2. `--server.port=9090` and packaged `server.port=8080`: CLI wins unless command-line properties were disabled.
3. Health endpoint exists but is not included in web exposure: no remote route through that technology.
4. Readiness depends on one shared database and it fails: all instances may leave traffic, depending on platform behavior.
5. `@SpringBootTest` contains a mock variant per class: context-cache fragmentation grows.
6. A payment POST times out: remote outcome is unknown.
7. Container limit is 512 MiB and heap max is 512 MiB: native/non-heap use can trigger kill.
8. Global lazy initialization is enabled: some bean errors move from startup to first use.
9. A property was removed in an upgrade: context may still start while old value has no effect.
10. A separate management port is healthy: main-port health is not proven.

TRY IT | Debugging exercises

1. **Missing controller:** application root is below the API package.
2. **Two clients:** missing-bean condition checks an implementation while user exposes an interface.
3. **Wrong timeout:** bare number interpreted in an unexpected default unit.
4. **Secret leak:** properties record is logged in a startup failure.
5. **400 becomes 500:** no handler for binding/validation contract.
6. **Duplicate orders:** idempotency stored only in an instance map.
7. **Retry storm:** gateway, service, and client each retry three times.
8. **Probe storm:** liveness calls a shared database.
9. **Slow tests:** every class changes profiles and mocks.
10. **Dropped rollout traffic:** platform grace is shorter than application drain.
11. **Migration race:** every pod runs a long non-transactional backfill.
12. **Public Actuator:** custom filter chain omitted management rules.
13. **Cardinality explosion:** metric tag contains raw order ID.
14. **Local-only success:** H2 is on the local/test classpath.
15. **Native failure:** reflection/resource behavior has no AOT hint.

Debugging corrections

Narrow and correct the scan/import boundary; align conditional types; add explicit units; redact secret-bearing types; centralize stable problem translation; persist idempotency durably with uniqueness; own one bounded retry layer; remove shared dependencies from liveness; reduce context variants; align readiness/drain/grace; coordinate migrations; explicitly secure management; replace IDs with bounded tags; test the target database; and add supported runtime hints or remove unnecessary dynamic behavior.

Small coding and design tasks

1. Create an immutable validated properties record.
2. Write an `ApplicationContextRunner` back-off test.
3. Return a problem detail for missing order.
4. Validate nested request lines.
5. Implement a deterministic cursor codec with signature.
6. Model idempotency states as an enum and transition table.
7. Configure a `RestClient` from typed properties.
8. Classify provider errors without leaking bodies.
9. Create a bounded health indicator.
10. Record accepted/rejected order counters with safe tags.
11. Add trace correlation to structured logs.
12. Write a readiness policy for required/optional dependencies.
13. Test graceful shutdown with one slow request.
14. Build a random-port endpoint test.
15. Add a target-database migration smoke test.
16. Build and inspect an OCI image.
17. Generate build information and expose only safe fields.
18. Compare startup steps before/after removing a starter.
19. Create a major-upgrade checklist.
20. Write a production 404 diagnostic script/runbook.

Interview follow-ups

1. Which Boot defaults would you never rely on without verification?
2. When should startup fail rather than mark readiness false?
3. How does back-off preserve application control?
4. Which configuration values may safely have defaults?
5. How would you rotate credentials at runtime?
6. When is a custom starter excessive abstraction?
7. How do you prevent JSON changes from breaking clients?
8. What does an idempotency key not solve?
9. Where should retry ownership live?
10. Which dependencies belong in readiness?
11. How do you secure heap-dump access?
12. What makes a metric tag unsafe?
13. How do you prove graceful shutdown?
14. When is a full-context test mandatory?
15. How do target containers still differ from production?
16. Why can a larger pool reduce throughput?
17. When should migrations run outside the app?
18. What justifies native image adoption?
19. How do you canary a framework upgrade?
20. Which evidence do you collect before restart?

Final readiness assessment

You are ready when you can, without notes:

- build and explain a packaged Boot service;
- trace startup and one condition report;
- predict configuration precedence from origins;
- design typed validated properties without leaking secrets;
- define stable HTTP, error, pagination, and idempotency contracts;
- budget outbound timeouts and retries;
- separate liveness, readiness, diagnostics, and graceful shutdown;
- select the smallest useful test boundary;
- explain artifact/image/resource behavior;
- diagnose startup, latency, memory, and rollout incidents from evidence.

If two areas remain weak, repeat their chapter exercises and one cumulative assessment before moving to Spring Data.

Cross-book boundaries

- Container, proxies, AOP, transactions, MVC internals -> **SD 04 - Spring Framework**.
- Maven/Gradle dependency and plugin depth -> **JAVA 03 - Maven and Gradle**.
- MySQL plans, locks, isolation, indexes -> **SD 02 - MySQL**.
- JPA mappings, persistence context, fetching -> **SD 03 - Hibernate and JPA**.
- Spring Security depth -> **SD 10 - Spring Ecosystem Extensions** or the security track.
- Spring Data repositories -> **SD 06 - Spring Data**.
- Kafka workflows -> **SD 09 - Apache Kafka and Spring Kafka**.
- JVM memory/GC/diagnostics -> **JAVA 06/JAVA 08**.
- Distributed consistency and system design -> **SD 14**.

Primary sources

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/reference/
- System Requirements: https://docs.spring.io/spring-boot/system-requirements.html
- Build Systems and Starters: https://docs.spring.io/spring-boot/reference/using/build-systems.html
- Auto-configuration: https://docs.spring.io/spring-boot/reference/using/auto-configuration.html
- Custom Auto-configuration: https://docs.spring.io/spring-boot/reference/features/developing-auto-configuration.html
- Externalized Configuration: https://docs.spring.io/spring-boot/reference/features/external-config.html
- Testing: https://docs.spring.io/spring-boot/reference/testing/
- Actuator Endpoints and Probes: https://docs.spring.io/spring-boot/reference/actuator/endpoints.html
- Container Images: https://docs.spring.io/spring-boot/reference/packaging/container-images/
- Spring Boot 4.1 Release Notes: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.1-Release-Notes

Validate release-specific behavior against the documentation for the Boot line deployed by your organization.

SDE-2 CORE CHAPTER

Chapter 24 - Java 21 Boot Runtime Companion

This dependency-free executable model validates property precedence and origin, conditional defaults and user back-off, application availability transitions, deadline allocation, and idempotency states; the Maven fixture separately executes real Spring Boot 4.1 behavior.

JAVA (continued 1/10)

```java
import java.time.Duration;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.Set;

/**
 * Dependency-free Java 21 companion for Spring Boot interview reasoning.
 *
 * <p>This is not a Boot replacement. It makes five decisions explicit:
 * configuration precedence, conditional defaults, availability transitions,
 * deadline allocation, and idempotent request claiming.</p>
 */
public final class SpringBootInterviewCompanion {
    private SpringBootInterviewCompanion() {
    }

    public static void main(String[] args) {
        validatesPropertyPrecedenceAndOrigin();
        validatesAutoConfigurationBackOff();
        validatesAvailabilityTransitions();
        validatesDeadlineBudgets();
        validatesIdempotencyClaims();
        System.out.println("SpringBootInterviewCompanion checks passed");
    }

    private static void validatesPropertyPrecedenceAndOrigin() {
        PropertyResolver resolver = new PropertyResolver(List.of(
                new PropertySource("packaged application.yml", 10,
                        Map.of("server.port", "8080", "payment.timeout", "3s")),
                new PropertySource("external production.yml", 20,
```

JAVA (continued 2/10)

```
                        Map.of("payment.timeout", "2s")),
                new PropertySource("environment", 30,
                        Map.of("server.port", "9090")),
                new PropertySource("command line", 40,
                        Map.of("payment.timeout", "750ms"))));

        ResolvedProperty port = resolver.required("server.port");
        require(port.value().equals("9090"), "environment must override packaged port");
        require(port.origin().equals("environment"), "port origin must be retained");

        ResolvedProperty timeout = resolver.required("payment.timeout");
        require(timeout.value().equals("750ms"), "command line must win");
        require(timeout.origin().equals("command line"), "timeout origin must be retained");
        require(resolver.resolve("missing").isEmpty(), "missing property must remain absent");
    }

    private static void validatesAutoConfigurationBackOff() {
        AutoConfigurationSpec specification = new AutoConfigurationSpec(
                "PaymentClientAutoConfiguration",
                "PaymentClient",
                Set.of("RestClient"),
                "payment.enabled");

        RuntimeInputs defaultInputs = new RuntimeInputs(
                Set.of("RestClient", "PaymentClient"),
                Set.of(),
                Map.of("payment.enabled", "true"));
        ConditionOutcome defaultOutcome = specification.evaluate(defaultInputs);
        require(defaultOutcome.matched(), "default must match when prerequisites exist");
        require(defaultOutcome.action().equals("register PaymentClient"),
                "default must register missing bean");

        RuntimeInputs userOverride = new RuntimeInputs(
                Set.of("RestClient", "PaymentClient"),
                Set.of("PaymentClient"),
                Map.of("payment.enabled", "true"));
```

JAVA (continued 3/10)

```
        ConditionOutcome overrideOutcome = specification.evaluate(userOverride);
        require(overrideOutcome.matched(), "configuration can match while bean backs off");
        require(overrideOutcome.action().equals("back off for user PaymentClient"),
                "user bean must win");

        RuntimeInputs disabled = new RuntimeInputs(
                Set.of("RestClient", "PaymentClient"),
                Set.of(),
                Map.of("payment.enabled", "false"));
        require(!specification.evaluate(disabled).matched(), "disabled property must skip");

        RuntimeInputs missingClass = new RuntimeInputs(
                Set.of("PaymentClient"),
                Set.of(),
                Map.of("payment.enabled", "true"));
        require(!specification.evaluate(missingClass).matched(),
                "missing class prerequisite must skip");
    }

    private static void validatesAvailabilityTransitions() {
        AvailabilityMachine machine = new AvailabilityMachine();
        require(machine.state() == AvailabilityState.STARTING, "initial state");
        machine.contextRefreshed();
        require(machine.state() == AvailabilityState.STARTED_NOT_READY,
                "refresh does not imply readiness");
        machine.startupWorkCompleted();
        require(machine.acceptsTraffic(), "ready state accepts traffic");
        machine.shutdownRequested();
        require(!machine.acceptsTraffic(), "shutdown refuses new traffic");
        machine.drainCompleted();
        require(machine.state() == AvailabilityState.STOPPED, "drain completes shutdown");

        boolean rejected = false;
        try {
            new AvailabilityMachine().startupWorkCompleted();
        } catch (IllegalStateException expected) {
```

JAVA (continued 4/10)

```
            rejected = true;
        }
        require(rejected, "invalid readiness transition must fail");
    }

    private static void validatesDeadlineBudgets() {
        DeadlineBudget budget = new DeadlineBudget(Duration.ofSeconds(2));
        budget.allocate("database", Duration.ofMillis(350));
        budget.allocate("payment", Duration.ofMillis(900));
        budget.allocate("response margin", Duration.ofMillis(300));
        require(budget.remaining().equals(Duration.ofMillis(450)), "remaining budget");

        boolean rejected = false;
        try {
            budget.allocate("retry", Duration.ofMillis(600));
        } catch (IllegalArgumentException expected) {
            rejected = true;
        }
        require(rejected, "nested work must not exceed caller deadline");
    }

    private static void validatesIdempotencyClaims() {
        IdempotencyStore store = new IdempotencyStore();
        Claim first = store.claim("key-1", "hash-A");
        require(first.status() == ClaimStatus.NEW, "first claim must be new");

        Claim concurrent = store.claim("key-1", "hash-A");
        require(concurrent.status() == ClaimStatus.IN_PROGRESS,
                "same request must observe in-progress state");

        store.complete("key-1", "order-42");
        Claim replay = store.claim("key-1", "hash-A");
        require(replay.status() == ClaimStatus.REPLAY, "completed request must replay");
        require(replay.result().orElseThrow().equals("order-42"), "stored result must return");

        Claim conflict = store.claim("key-1", "hash-B");
```

JAVA (continued 5/10)

```
        require(conflict.status() == ClaimStatus.CONFLICT,
                "same key with another request must conflict");
    }

    private static void require(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    record PropertySource(String name, int precedence, Map<String, String> values) {
        PropertySource {
            Objects.requireNonNull(name);
            values = Map.copyOf(values);
        }
    }

    record ResolvedProperty(String name, String value, String origin, int precedence) {
    }

    static final class PropertyResolver {
        private final List<PropertySource> sources;

        PropertyResolver(List<PropertySource> sources) {
            this.sources = new ArrayList<>(sources);
            this.sources.sort(Comparator.comparingInt(PropertySource::precedence).reversed());
        }

        Optional<ResolvedProperty> resolve(String name) {
            return sources.stream()
                    .filter(source -> source.values().containsKey(name))
                    .findFirst()
                    .map(source -> new ResolvedProperty(
                            name,
                            source.values().get(name),
                            source.name(),
```

JAVA (continued 6/10)

```
                            source.precedence()));
        }

        ResolvedProperty required(String name) {
            return resolve(name).orElseThrow(() ->
                    new IllegalArgumentException("Missing property: " + name));
        }
    }

    record RuntimeInputs(Set<String> classes,
                         Set<String> beans,
                         Map<String, String> properties) {
        RuntimeInputs {
            classes = Set.copyOf(classes);
            beans = Set.copyOf(beans);
            properties = Map.copyOf(properties);
        }
    }

    record ConditionOutcome(boolean matched, List<String> reasons, String action) {
        ConditionOutcome {
            reasons = List.copyOf(reasons);
        }
    }

    record AutoConfigurationSpec(String name,
                                 String beanType,
                                 Set<String> requiredClasses,
                                 String enabledProperty) {
        AutoConfigurationSpec {
            requiredClasses = Set.copyOf(requiredClasses);
        }

        ConditionOutcome evaluate(RuntimeInputs inputs) {
            List<String> reasons = new ArrayList<>();
            Set<String> missing = new java.util.LinkedHashSet<>(requiredClasses);
```

JAVA (continued 7/10)

```
            missing.removeAll(inputs.classes());
            if (!missing.isEmpty()) {
                reasons.add("missing classes " + missing);
                return new ConditionOutcome(false, reasons, "skip " + name);
            }
            reasons.add("required classes present");

            boolean enabled = Boolean.parseBoolean(
                    inputs.properties().getOrDefault(enabledProperty, "true"));
            if (!enabled) {
                reasons.add(enabledProperty + " is false");
                return new ConditionOutcome(false, reasons, "skip " + name);
            }
            reasons.add(enabledProperty + " is true");

            if (inputs.beans().contains(beanType)) {
                reasons.add("user bean exists");
                return new ConditionOutcome(true, reasons, "back off for user " + beanType);
            }
            reasons.add("bean is missing");
            return new ConditionOutcome(true, reasons, "register " + beanType);
        }
    }

    enum AvailabilityState {
        STARTING,
        STARTED_NOT_READY,
        ACCEPTING_TRAFFIC,
        DRAINING,
        STOPPED
    }

    static final class AvailabilityMachine {
        private AvailabilityState state = AvailabilityState.STARTING;

        AvailabilityState state() {
```

JAVA (continued 8/10)

```
            return state;
        }

        boolean acceptsTraffic() {
            return state == AvailabilityState.ACCEPTING_TRAFFIC;
        }

        void contextRefreshed() {
            transition(AvailabilityState.STARTING, AvailabilityState.STARTED_NOT_READY);
        }

        void startupWorkCompleted() {
            transition(AvailabilityState.STARTED_NOT_READY, AvailabilityState.ACCEPTING_TRAFFIC);
        }

        void shutdownRequested() {
            transition(AvailabilityState.ACCEPTING_TRAFFIC, AvailabilityState.DRAINING);
        }

        void drainCompleted() {
            transition(AvailabilityState.DRAINING, AvailabilityState.STOPPED);
        }

        private void transition(AvailabilityState expected, AvailabilityState next) {
            if (state != expected) {
                throw new IllegalStateException(
                        "Expected " + expected + " but was " + state);
            }
            state = next;
        }
    }

    static final class DeadlineBudget {
        private final Duration total;
        private final Map<String, Duration> allocations = new LinkedHashMap<>();
```

JAVA (continued 9/10)

```
        DeadlineBudget(Duration total) {
            if (total.isNegative() || total.isZero()) {
                throw new IllegalArgumentException("total must be positive");
            }
            this.total = total;
        }

        void allocate(String name, Duration duration) {
            if (duration.isNegative() || duration.isZero()) {
                throw new IllegalArgumentException("allocation must be positive");
            }
            if (duration.compareTo(remaining()) > 0) {
                throw new IllegalArgumentException("allocation exceeds remaining deadline");
            }
            allocations.put(name, duration);
        }

        Duration remaining() {
            Duration used = allocations.values().stream()
                    .reduce(Duration.ZERO, Duration::plus);
            return total.minus(used);
        }
    }

    enum ClaimStatus {
        NEW,
        IN_PROGRESS,
        REPLAY,
        CONFLICT
    }

    record Claim(ClaimStatus status, Optional<String> result) {
        static Claim of(ClaimStatus status) {
            return new Claim(status, Optional.empty());
```

JAVA (continued 10/10)

```
        }
    }

    record StoredRequest(String requestHash, Optional<String> result) {
        StoredRequest {
            Objects.requireNonNull(requestHash);
            Objects.requireNonNull(result);
        }
    }

    static final class IdempotencyStore {
        private final Map<String, StoredRequest> requests = new HashMap<>();

        Claim claim(String key, String requestHash) {
            StoredRequest existing = requests.get(key);
            if (existing == null) {
                requests.put(key, new StoredRequest(requestHash, Optional.empty()));
                return Claim.of(ClaimStatus.NEW);
            }
            if (!existing.requestHash().equals(requestHash)) {
                return Claim.of(ClaimStatus.CONFLICT);
            }
            return existing.result()
                    .map(result -> new Claim(ClaimStatus.REPLAY, Optional.of(result)))
                    .orElseGet(() -> Claim.of(ClaimStatus.IN_PROGRESS));
        }

        void complete(String key, String result) {
            StoredRequest existing = Optional.ofNullable(requests.get(key))
                    .orElseThrow(() -> new IllegalArgumentException("unknown key"));
            requests.put(key, new StoredRequest(
                    existing.requestHash(), Optional.of(result)));
        }
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: first application, project anatomy, startup, structure, build, and typed configuration
- Interview Core: conditions, back-off, REST, HTTP clients, data, Actuator, observability, and focused tests
- SDE-2 Follow-up: idempotency, retries, migration safety, probes, shutdown, containers, upgrades, security, and incidents
- Readiness: complete 28 answered interview rounds, five cumulative assessments, and both executable validation layers

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: FW 03 - Spring Framework for Java Engineers

Series index

Next book: FW 05 - Spring Boot and REST APIs

TRY IT | Completion check

[ ] I can recognize the core patterns without being told the data structure or algorithm.
[ ] I can implement the representative Java templates from memory.
[ ] I can explain why each implementation is correct.
[ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
[ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 04 of 12 - Study Step FW-04. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.